

Lab 4: ROM-based Game Access Control on FPGA

Tru Nguyen

PSID: 1911032

ECE 6370

1. Introduction

This project is the design of a game that can be played by two players on the DE0-CV FPGA board. Each player has one set of four slide switches on the board corresponding to a 4-bit binary number. Player 1 switches are the four rightmost switches (SW3-SW0) which SW0 is the least significant bit. Player 2 switches are the four leftmost switches (SW9-SW6) which SW6 is the least significant bit. Player 1 and Player 2's numbers are displayed on the rightmost 7-segment display and the second to leftmost display in hexadecimal value respectively. The sum of the two players' numbers is displayed on the third 7-segment display from the right in hexadecimal value. The leftmost LED is lit if the sum of the two numbers equal 4'b1111 or its right-adjacent LED is lit if otherwise.

Players have their own LOAD button that will display their respective number on the switches on the 7-segment displays. Before any player can load their number, the system password must be entered using Player 2's set of switches. If the password is successfully entered, the rightmost LED is lit up indicating logged-in status or the left-adjacent LED is lit if a correct password has not been entered. Once logged in and the password LOAD button is pressed again, players will be logged out. They cannot load their numbers until logged in again. The RESET button will log players out and set the board back to default settings.

The rule of the game is that Player 1 sets their number using the switches then Player 2 needs to set a number (one-time only) that the sum of both numbers equals 4'b1111 without looking at Player 1's switches. If Player 2 gets the right number, they win the round. If Player 2 gets the wrong number, Player 1 wins the round. The players switch roles after every round as Player 2 now sets a number for Player 1 to match.

In Lab 3, the system is enhanced by replacing Player 2 with a hardware-based true Random Number Generator (RNG) that produces a 4-bit value (0–15). The RNG is implemented using a 4-bit counter driven by the 50 MHz clock. A dedicated button is used to generate the random number: when the player presses and holds the button, the counter increments continuously, and when the button is released, the current counter value is captured and displayed as the target number. This button is intentionally not debounced so that the duration of the press determines the randomness.

The password protection and load functionality from Lab 2 are preserved. Before the correct password is entered, pressing either the random number generation button or the player LOAD button will have no effect. This is achieved by routing the RNG button through the access controller, which forces the signal inactive until authentication is successful. Once the password is verified, the RNG and load operations function normally.

Additionally, a timer feature is introduced to control gameplay. The game operates with a default countdown of 99 seconds, displayed in base-10 format on the 7-segment displays. Before password verification, the timer displays "00." After successful login, the timer is initialized to 99 seconds but does not begin counting until the GAME button is pressed. During this time, the player attempts to complete as many rounds as possible by selecting a number that, when added to the generated random number, equals 4'b1111.

When the timer reaches zero, the game ends and the player is no longer able to generate new random numbers or load inputs. Pressing the GAME button after timeout reloads the timer to 99 seconds, and pressing it again restarts the countdown. The logout functionality from Lab 2 is disabled in this version since the same button is repurposed as the GAME control.

Lab 4 introduces two additional implementation features while keeping the same user interface and interaction sequence. First, the timer control function is separated from the password authentication logic, and the original millisecond-based timer is replaced with a more cost-efficient design based on a Linear Feedback Shift Register (LFSR). Second, player authentication is modified so that the default password is stored in on-chip ROM and fetched during verification rather than being hardcoded in logic. The password entry sequence and game start/restart behavior remain consistent with Lab 3. As optional bonus features, the system supports logging out by pressing the Player LOAD button when the game is not in an active timed session, and after logout all gameplay functions (including RNG and timer activity) are disabled until the user logs in again. Additionally, the RNG's internal 4-bit counter can be replaced with a more cost-efficient 4-bit LFSR-based counter while maintaining the same press-and-hold capture behavior.

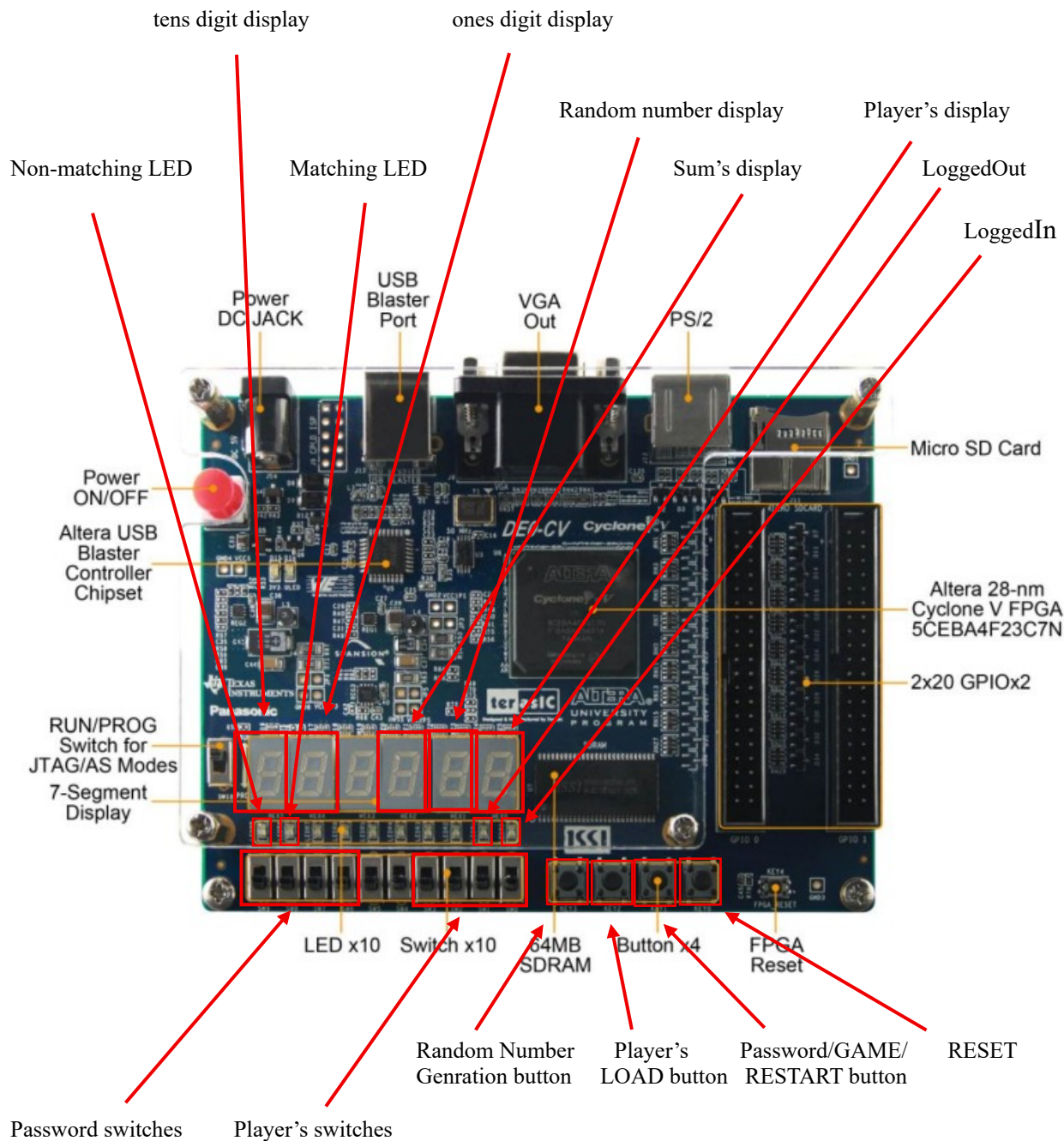


Figure 1: DE0-CV FPGA board with annotated Players' switches, displays, buttons and LEDs

2. System Architecture Design

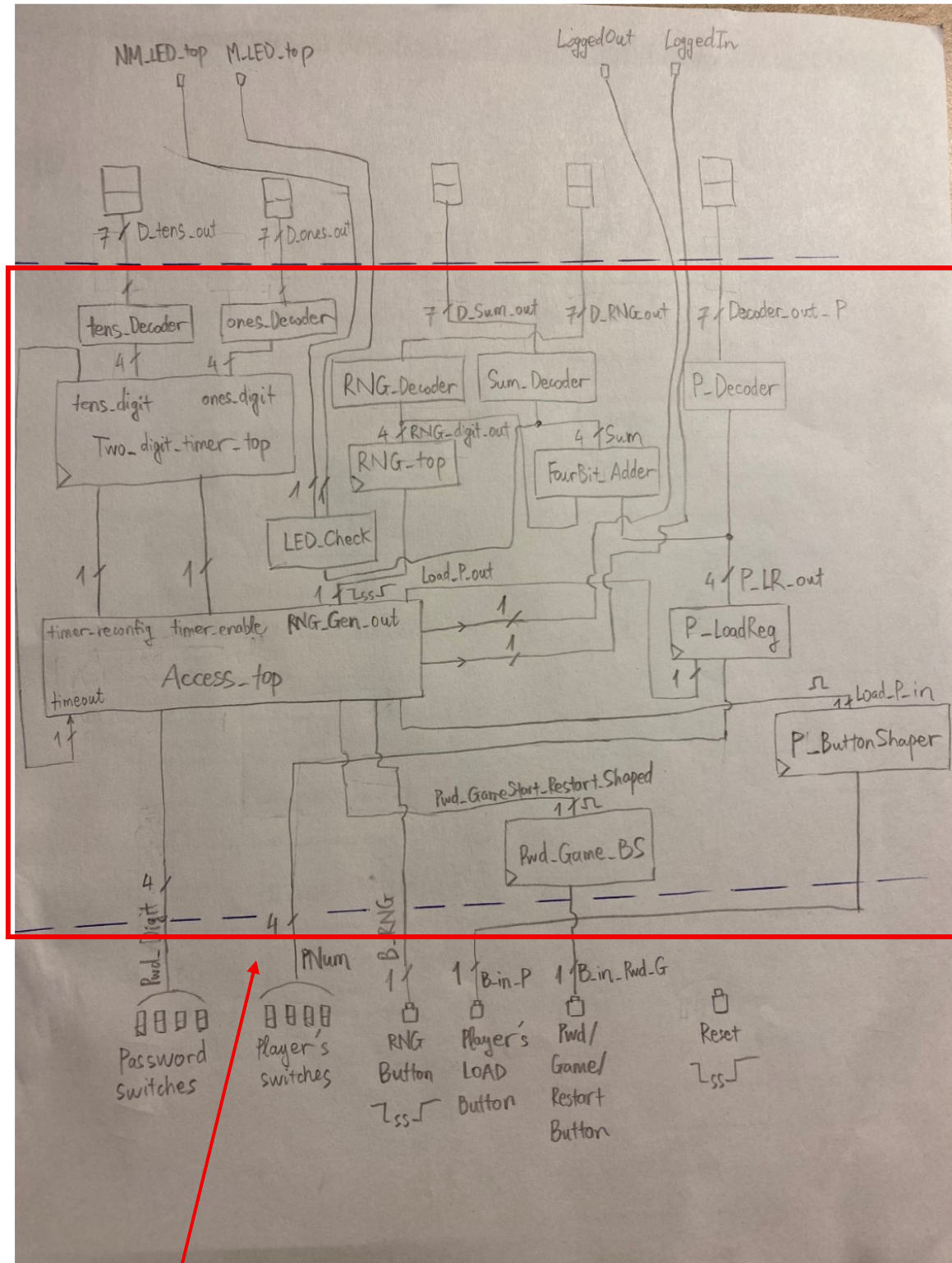
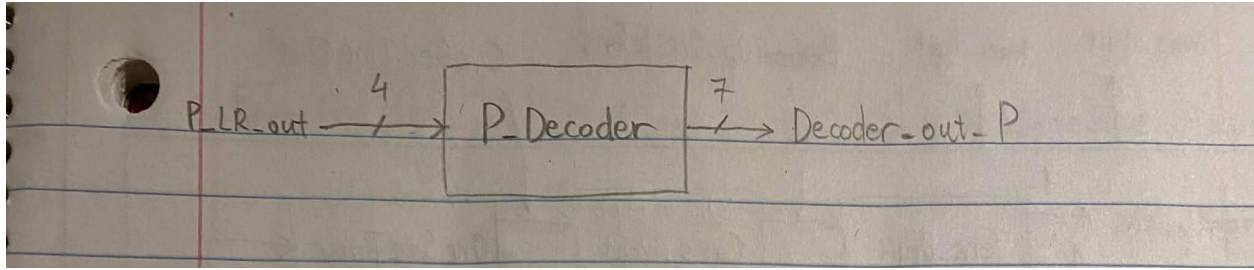


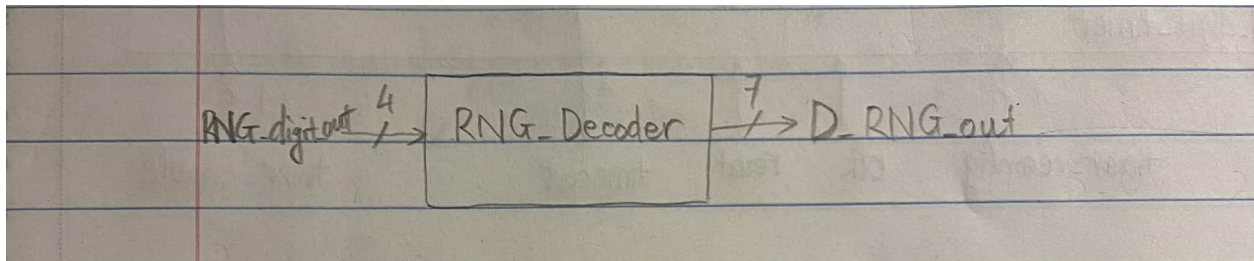
Figure 2: Top-level system architecture
Top-level module Lab2_NGUYEN_Tru

2.1. P_Decoder



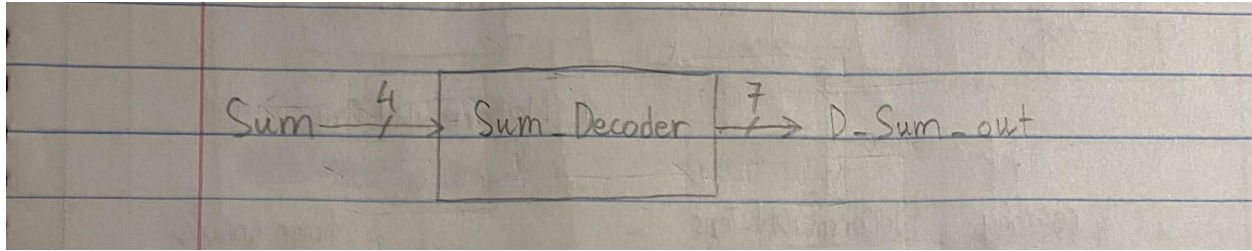
- **Function:** decodes the 4-bit binary number from Player's switches to respective hexadecimal value displayed on Player's display.
- **Input:**
[3:0] P_LR_out: Player's 4-bit binary number from switches.
- **Output:**
[6:0] Decoder_out_P: 7-bit value shows which indexed segment to be turned on and off on Player's display.

2.2. RNG_Decoder



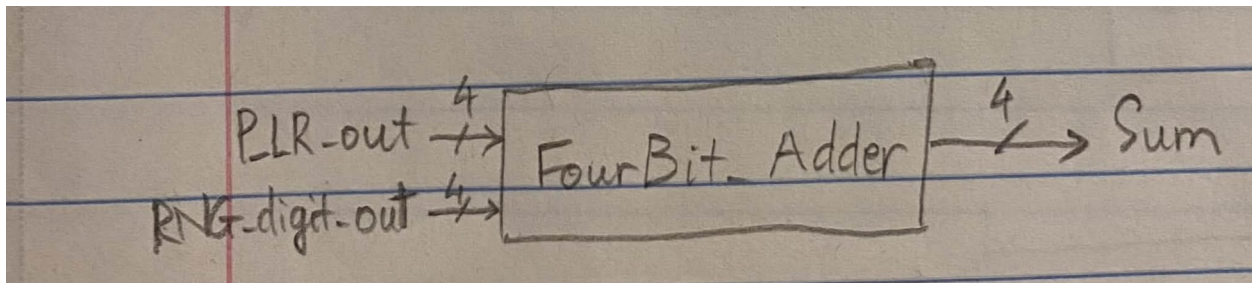
- **Function:** decodes the 4-bit binary number from the random number generator to respective hexadecimal value displayed on RNG's display.
- **Input:**
[3:0] RNG_digit_out: Randomly generated 4-bit binary number from RNG Button.
- **Output:**
[6:0] D_RNG_out_2: 7-bit value shows the randomly generated number on respective display.

2.3. Sum_Decoder



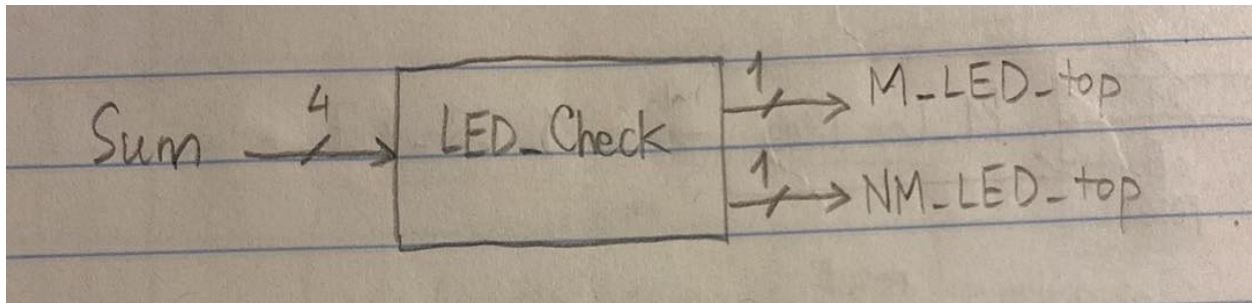
- **Function:** decodes the 4-bit output binary number from the FourBit_Adder module to respective hexadecimal value displayed on Sum display.
- **Input:**
[3:0] Sum: 4-bit binary sum of Player and the generated random 4-bit binary numbers.
- **Output:**
[6:0] D_Sum_out: 7-bit value shows which indexed segment to be turned on and off on Sum display.

2.4. FourBit_Adder



- **Function:** performs the sum of Player and the randomly generated s 4-bit binary numbers.
- **Input:**
[3:0] P_LR_out: Player's 4-bit binary number from switches held in P_LoadReg.
[3:0] RNG_digit_out: Randomly generated 4-bit binary random number.
- **Output:**
[3:0] Sum: holds the 4-bit binary sum of Player and the generated random 4-bit binary numbers.

2.5. LED_Check



- **Function:** verifies if the sum of Player 1 and 2's numbers equals 1111 (binary). If so, lights up the leftmost LED or the rightmost LED if otherwise.

- **Input:**

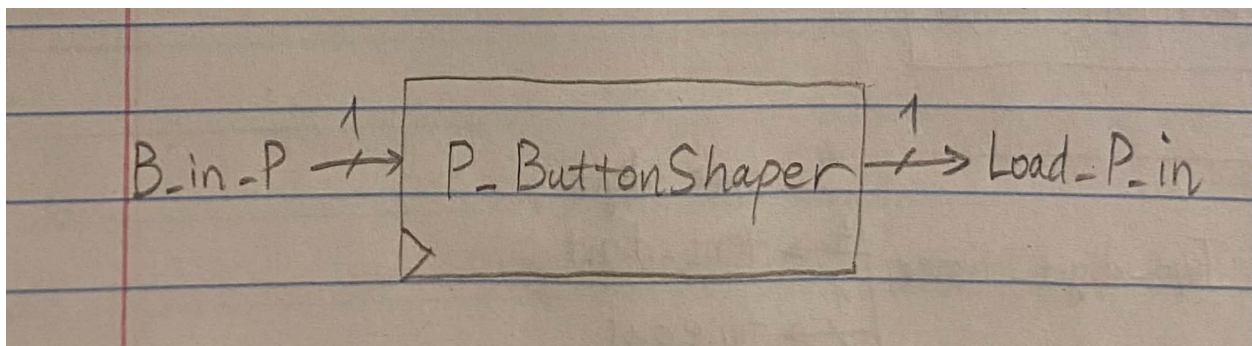
[3:0] Sum: holds the 4-bit binary sum of Player 1 and 2's 4-bit binary numbers.

- **Output:**

M_LED_top: drives the matching LED.

NM_LED_top: drives the non-matching LED.

2.6. P_ButtonShaper



- **Function:** Generates a single clock-cycle pulse when Player's button is pressed.

- **Input:**

B_in_P: raw Player's button push signal.

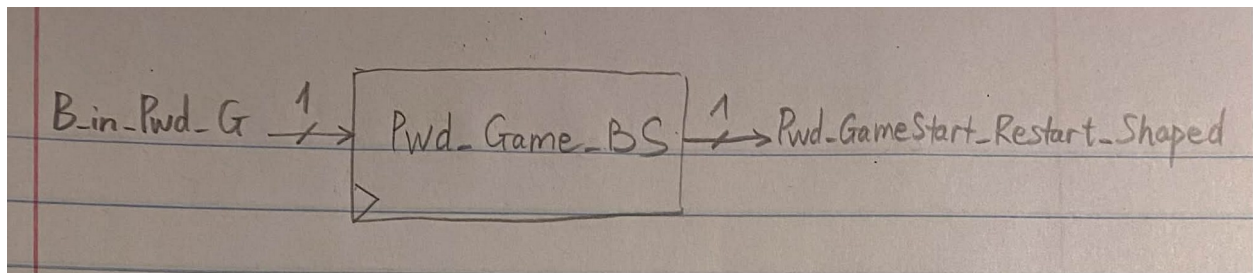
reset: active low reset.

clk: system clock.

- **Output:**

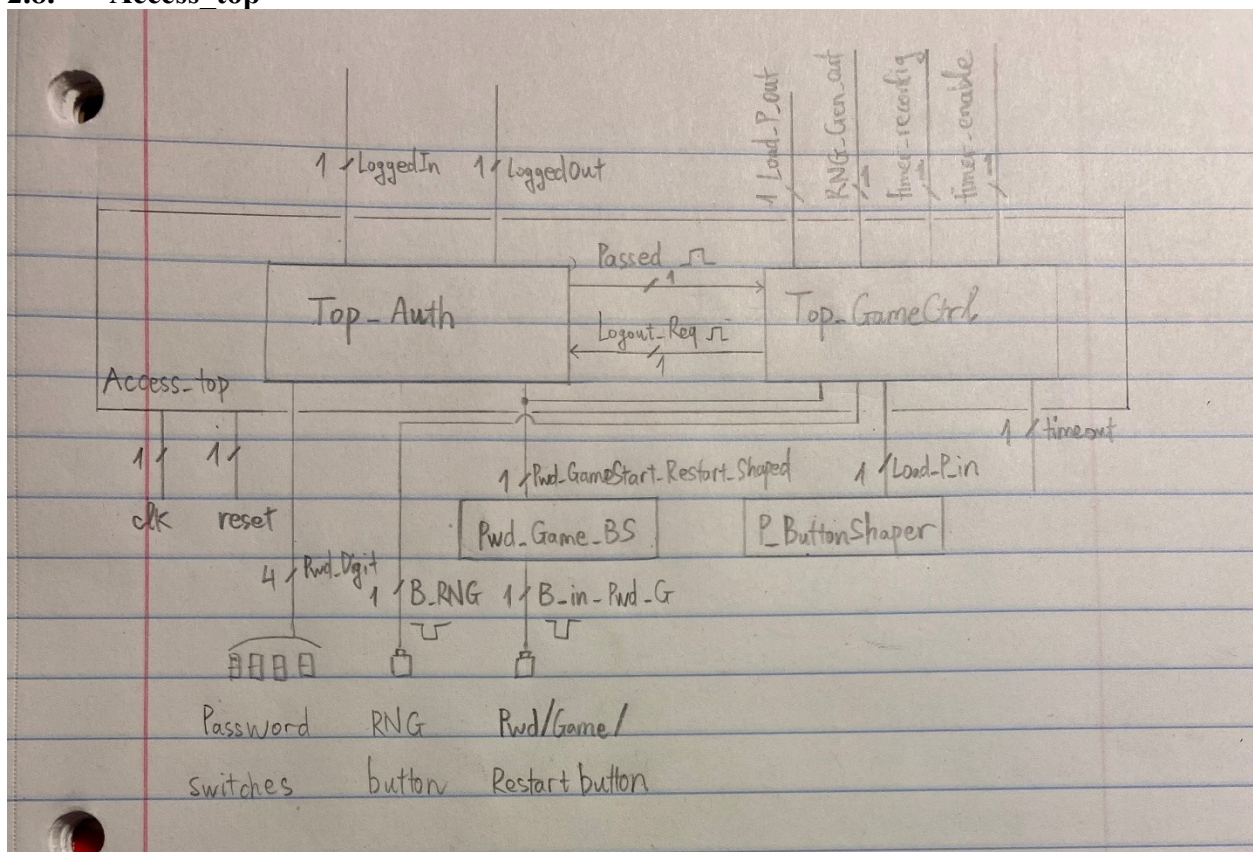
Load_P_in: Clean one-cycle pulse used to request loading Player's number.

2.7. Pwd_Game_BS



- **Function:** Generates a single clock-cycle pulse when password-enter button is pressed and to start or restart gameplay.
- **Input:**
 B_in_Pwd_G: raw password-enter/game start/game restart button push signal.
 reset: active low reset.
 clk: system clock.
- **Output:**
 Pwd_GameStart_Restart_Shaped: Clean one-cycle pulse sent to Access_top.

2.8. Access_top



- **Function:** Access_top is the top-level control and permission (authentication + gameplay gating) module
- **Input:**

Pwd_GameStart_Restart_Shaped: one-cycle pulse to enter a password digit.

[3:0] Pwd_Digit: the 4-bit password digit currently set on the switches.

Load_P_in: load request for Player's number.

B_RNG: raw button push signal to request a random number generation.

timeout: stops timer counting after time is up.

reset: active low reset.

clk: system clock.

- **Output:**

Load_P_out: Clean one-cycle pulse allowing Player's number to be loaded.

LoggedIn: LED indicator (user authenticated).

LoggedOut: LED indicator (user not authenticated).

timer_reconfig: reload digit 9 to both ones and tens display.

timer_enable: enable timer to start counting down.

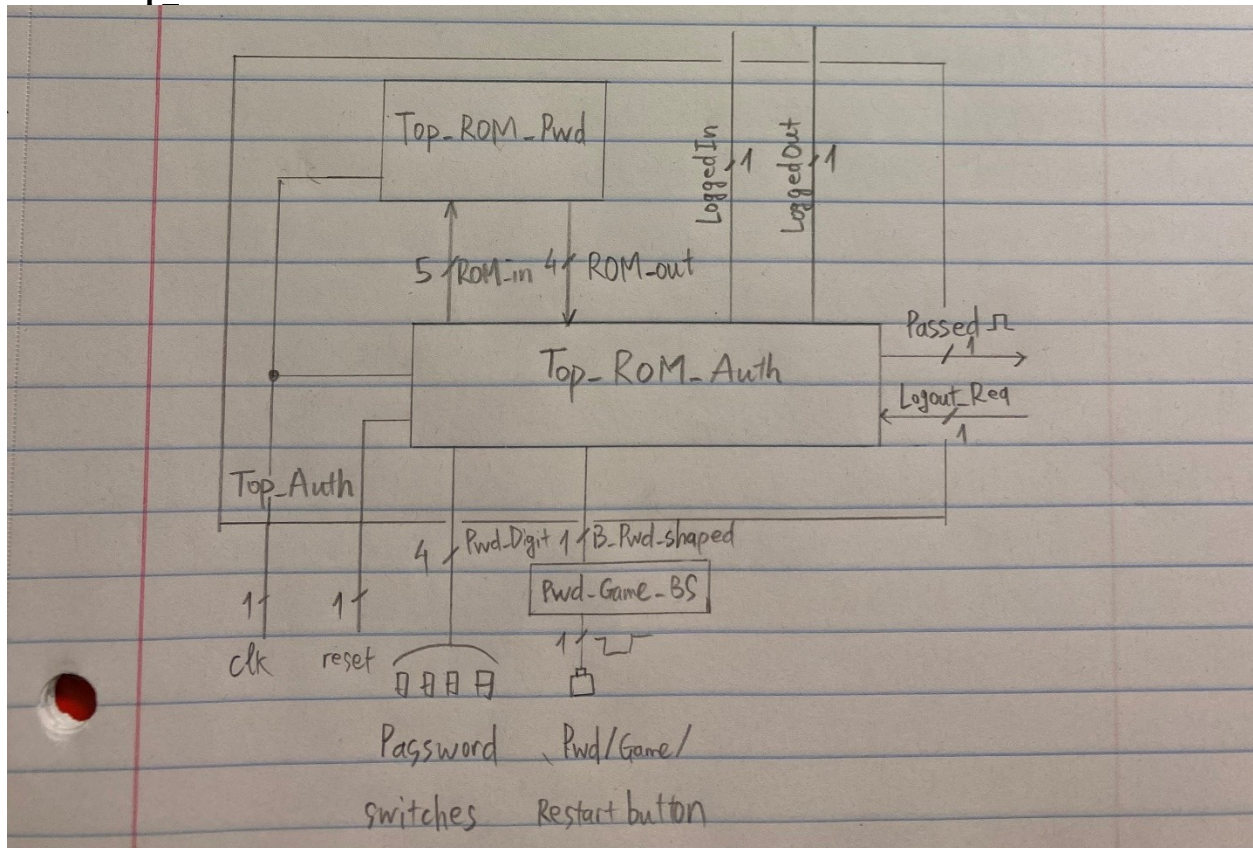
RNG_Gen_out: raw button push signal from the random number generation button allowed to pass to the RNG_top module.

- **Internal wires:**

Passed: internal "authentication passed" flag from Top_Auth to Top_GameCtrl. This is the key authorization signal used to enable/disable game functions.

Logout_Req: Internal "logout request" from Top_GameCtrl to Top_Auth. Used to clear the logged-in state when the Player requests logout (e.g., pressing LOAD when timer not running).

2.9. Top_GameCtrl



- **Function:** It generates the control signals that enable/disable gameplay functions—player load, RNG generation, and timer start/reload—based on the shaped GAMEPLAY button, RNG button, player LOAD button, the timer timeout flag, and whether authentication has passed (Passed). It also issues a Logout_Req pulse to the authentication logic when the user requests logout.

- **Input:**

B_GamePlay_Shaped: one-pulse GAMEPLAY control button. Used to start the timer/gameplay (transition into GAMEPLAY) and to restart/reconfigure the timer after game over.

B_RNG: raw RNG button input. During active gameplay, this input is routed to the RNG_top via RNG_Gen_out so the RNG_top can advance/capture values.

Load_P_in: Player LOAD request button. During active gameplay it is forwarded to Load_P_out to allow loading/capturing the player's number. When the game is not active, this button is repurposed to request logout.

timeout: timer expired indicator. When asserted, the controller disables any gameplay actions from the Player.

Passed: authentication status from the Top_Auth module. When Passed = 1, the user is logged in and the controller can proceed to configure/start the game. When Passed = 0, the system should remain in the initial “locked” behavior.

reset: active low reset.

clk: system clock.

- **Output:**

Load_P_out: gated/authorized load signal to the rest of the datapath. It mirrors Load_P_in only during the gameplay state; otherwise it is forced low.

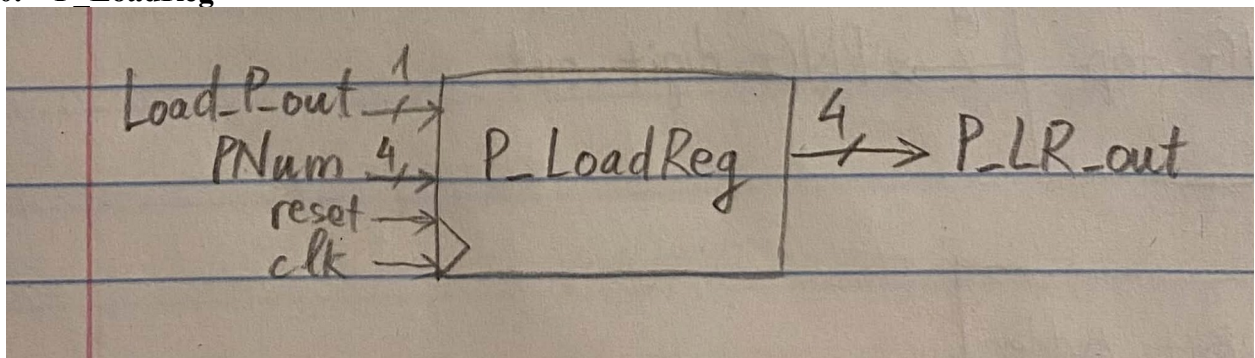
RNG_Gen_out: gated/authorized RNG control output. It mirrors B_RNG only during the gameplay state; otherwise it is forced low.

timer_reconfig: timer reinitialize/reload control. One-cycle assertion causes the timer to load the starting count (e.g., 99 seconds).

timer_enable: timer run control. Asserted during gameplay to enable the countdown; deasserted otherwise.

Logout_Req: Logout request pulse sent to the Top_Auth module.

2.10. P_LoadReg



- **Function:** stores Player's 4-bit number when enabled.

- **Input:**

[3:0] PNum: Player's 4-bit input number.

Load_P_out: Load enable signal.

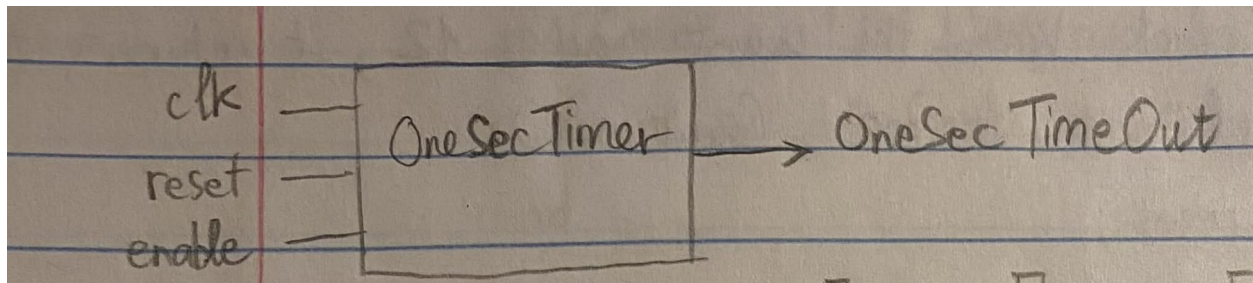
reset: active low reset.

clk: system clock.

- **Output:**

P_LR_out: Player's stored 4-bit value.

2.11. OneSecTimer



OneSecTimer module is the top-level timing module that generates a 1-second timeout signal by cascading three smaller timers. It uses a 1 ms timer, then counts those pulses to form 100 ms, and finally counts again to produce a 1-second pulse.

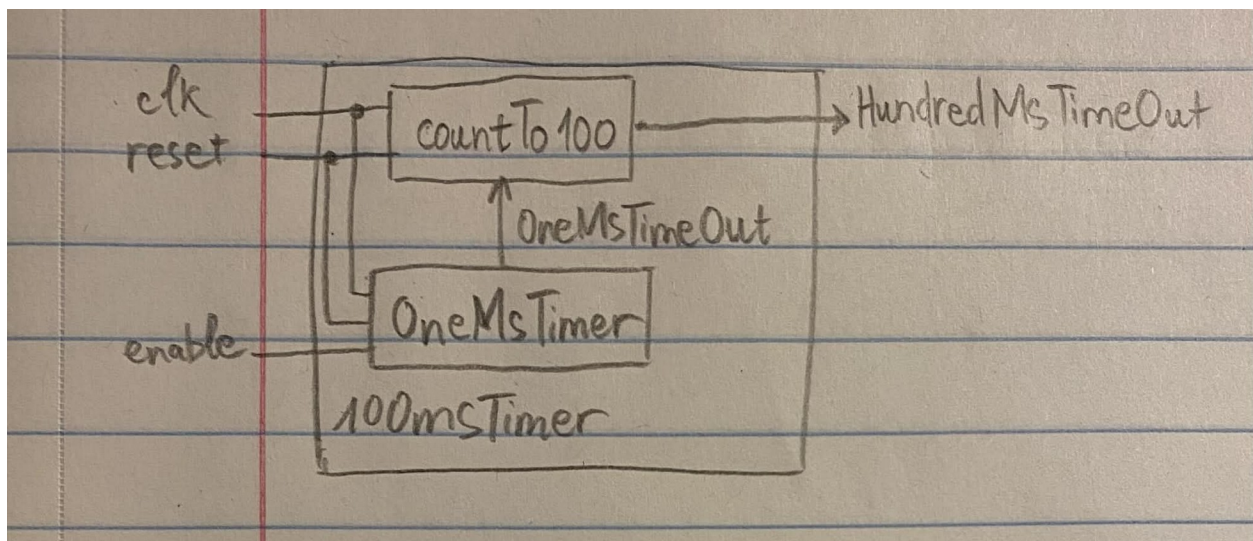
Inputs:

- enable — enables the timing operation
- clk — system clock
- reset — active-low reset (resets all counters)

Output:

- OneSecTimeOut — asserted for one clock cycle every 1 second

2.12. countTo100



countTo100 module counts incoming **1 ms timeout pulses** and generates a **100 ms timeout signal**. When the count reaches its limit, it resets and produces a one-cycle output pulse.
(Note: Counter size is reduced for simulation purposes.)

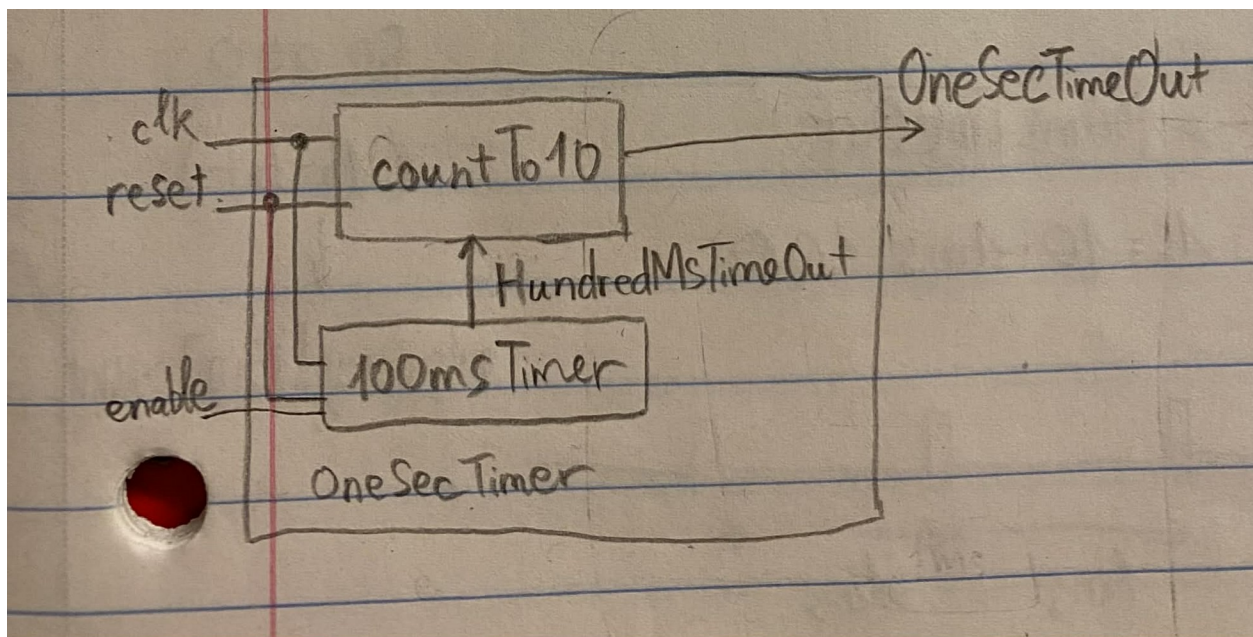
Inputs:

- OneMsTimeOut — input pulse every 1 ms
- clk — system clock
- reset — active-low reset

Outputs:

- HundredMsTimeOut — asserted for one clock cycle every 100 ms
- [1:0] Count_out — current counter value

2.13. countTo10



countTo10 module counts 100 ms timeout pulses and generates a 1 second timeout signal. Once the count limit is reached, it resets and asserts **OneSecTimeOut** for one clock cycle.
(Note: Counter size is reduced for simulation purposes.)

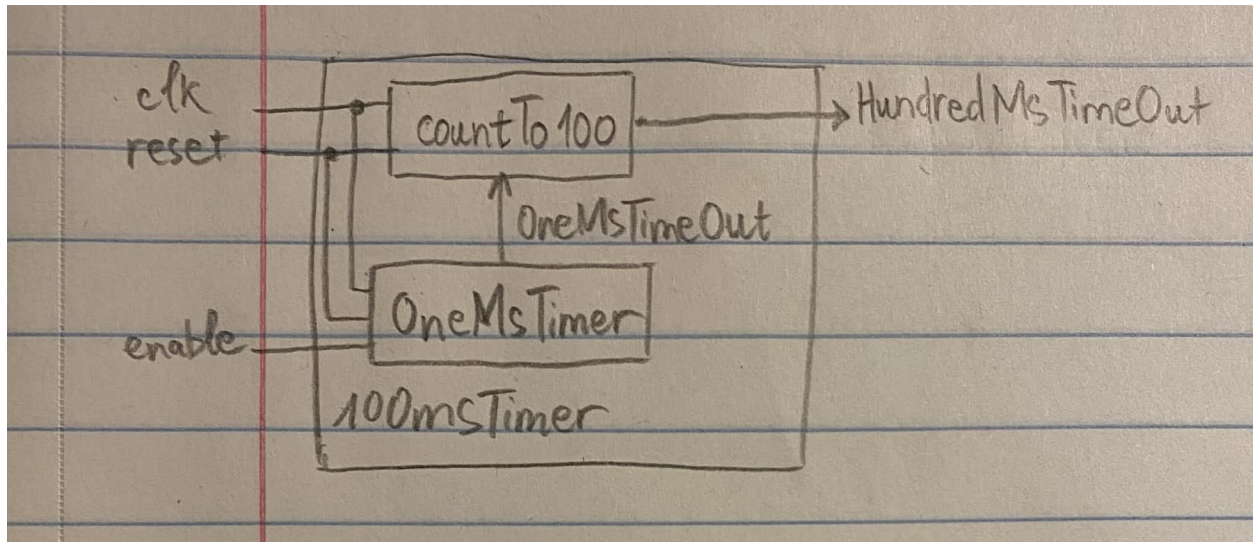
Inputs:

- HundredMsTimeOut — input pulse every 100 ms
- clk — system clock
- reset — active-low reset

Outputs:

- OneSecTimeOut — asserted for one clock cycle every 1 second
- [1:0] Count_out — current counter value

2.13 OneMsTimer



OneMsTimer module generates a 1 millisecond timeout pulse by counting clock cycles when enabled. When the internal counter reaches its maximum value, it resets and asserts OneMsTimeout for one clock cycle.

(Note: Counter size is reduced for simulation purposes.)

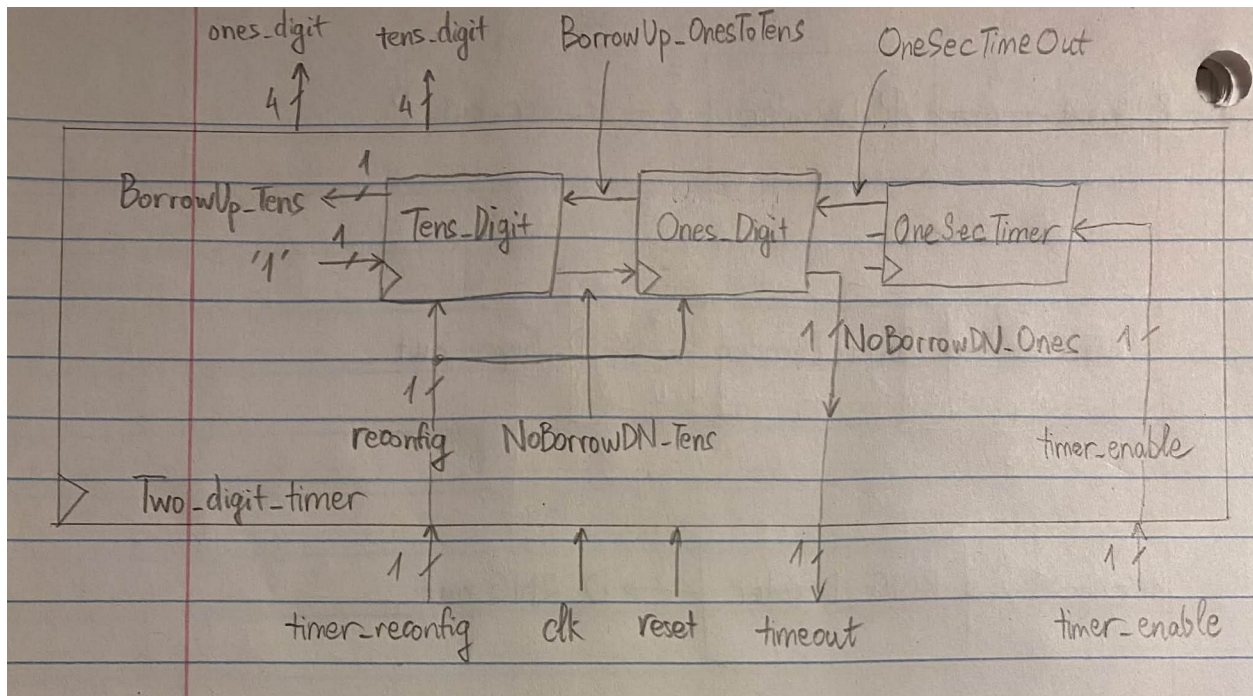
Inputs:

- enable — enables counting
- clk — system clock
- reset — active-low reset

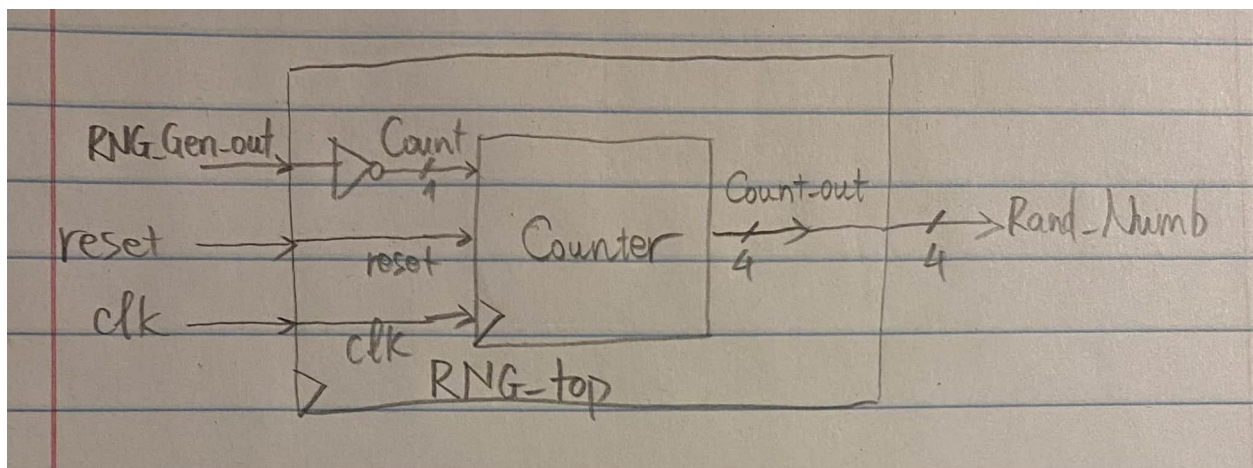
Outputs:

- OneMsTimeout — goes high for one clock cycle when count completes
- [2:0] Count_out — current counter value

2.14 Two Digit Timer



2.15 RNG_top



3. Simulation Results

3.1. Seven Segment Decoder



Figure 3: 7-segment decoder simulation results

The 7-segment decoder simulation shows all sixteen cases of a 4-bit binary input number from the FPGA's slide switches and corresponding 7-bit output numbers that display indexed segments on 7-segment displays. This test input number is declared decoder_in_s and test output number decoder_out_s.

Sixteen input test cases are from 0 to F (hex) and corresponding output cases are shown as follows (numbers are in binary):

```

decoder_in_s = 0000; decoder_out_s = 1000000
decoder_in_s = 0001; decoder_out_s = 1111001
decoder_in_s = 0010; decoder_out_s = 0100100
decoder_in_s = 0011; decoder_out_s = 0110000
decoder_in_s = 0100; decoder_out_s = 0011001
decoder_in_s = 0101; decoder_out_s = 0010010
decoder_in_s = 0110; decoder_out_s = 0000010
decoder_in_s = 0111; decoder_out_s = 1111000
decoder_in_s = 1000; decoder_out_s = 0000000
decoder_in_s = 1001; decoder_out_s = 0010000
decoder_in_s = 1010; decoder_out_s = 0001000
decoder_in_s = 1011; decoder_out_s = 0000011
decoder_in_s = 1100; decoder_out_s = 0100111
decoder_in_s = 1101; decoder_out_s = 0100001

```



```

decoder_in_s = 1110; decoder_out_s = 0000110
decoder_in_s = 1111; decoder_out_s = 0001110
default; decoder_out_s = 7'b1000000

```

3.2. Four Bit Adder

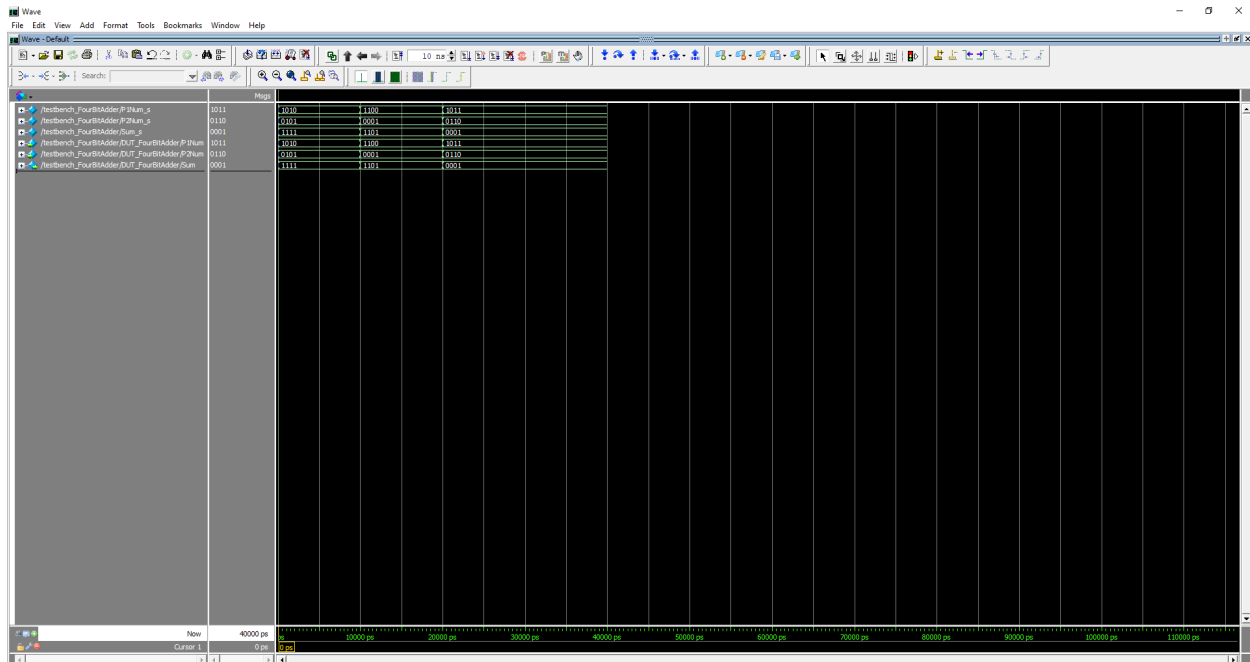


Figure 4: 4-bit adder simulation results

The 4-bit adder simulation shows three sets of test 4-bit input binary numbers from the FPGA's slide switches and corresponding sum of each set. These three sets of test input numbers include a case of matching number, which the sum equals 1111 in binary, and two cases of non-matching numbers. One of the two non-matching cases shows an overflow binary addition, which the output only includes the four least significant bits. These two test input numbers are declared P1Num_s and P2Num_s while the output number is Sum_s.

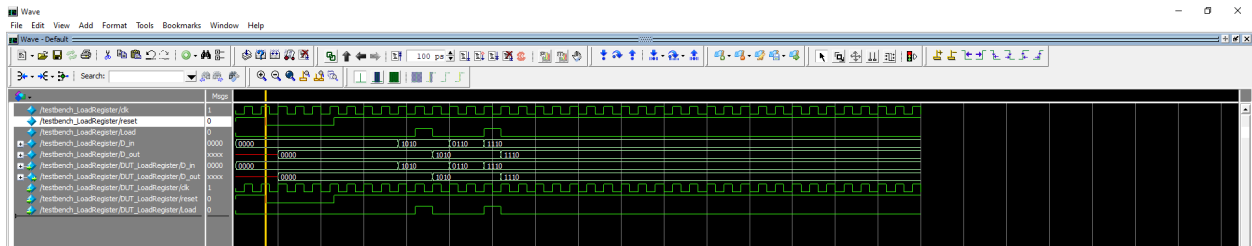
Three sets of input test cases and corresponding output numbers are as follows (numbers are in binary):

P1Num_s = 1010, P2Num_s = 0101, Sum_s = 1111 (matching case)

P1Num_s = 1100, P2Num_s = 0001, Sum_s = 1101 (non-matching case)

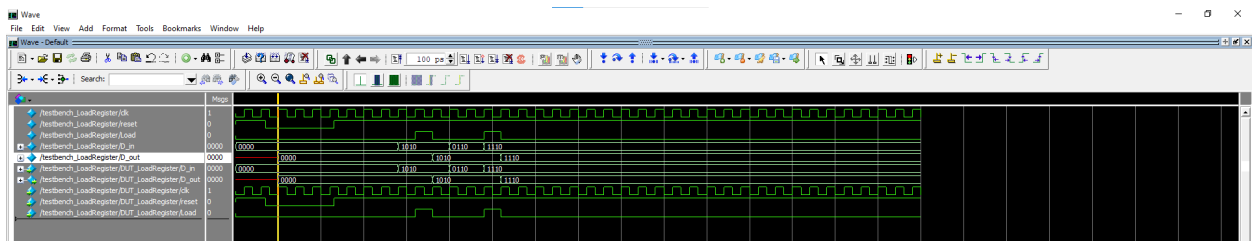
P1Num_s = 1011, P2Num_s = 0110, Sum_s = (1) 0001 (non-matching overflow case)

3.3. Load Register

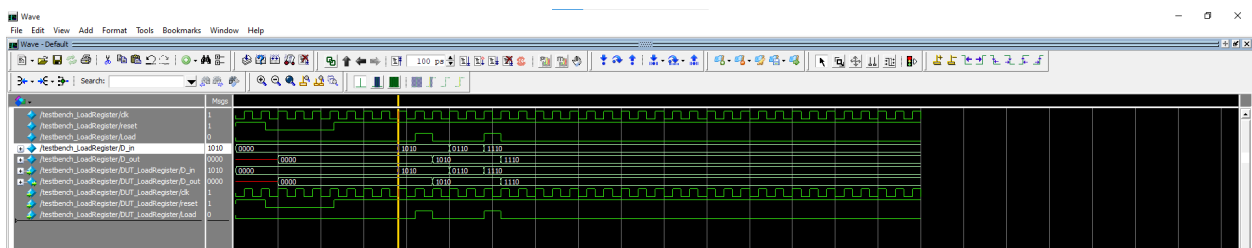


D_in is initially loaded with 4'b0000. While reset is high initially, D_out isn't loaded with the value of D_in and doesn't store anything initially.

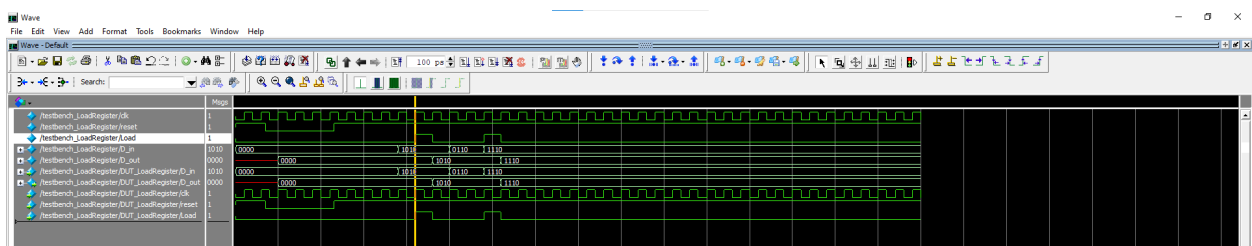
The figure above shows when reset button is pushed (active low).



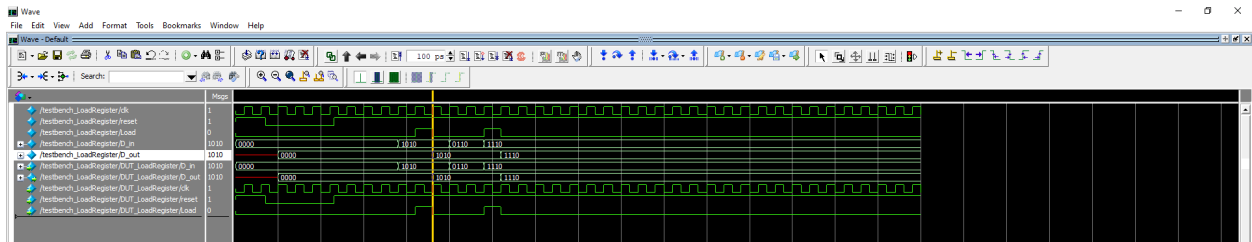
D_out is loaded with D_in value of 4'b0000 at the following rising clk edge after reset button is pushed (active low).



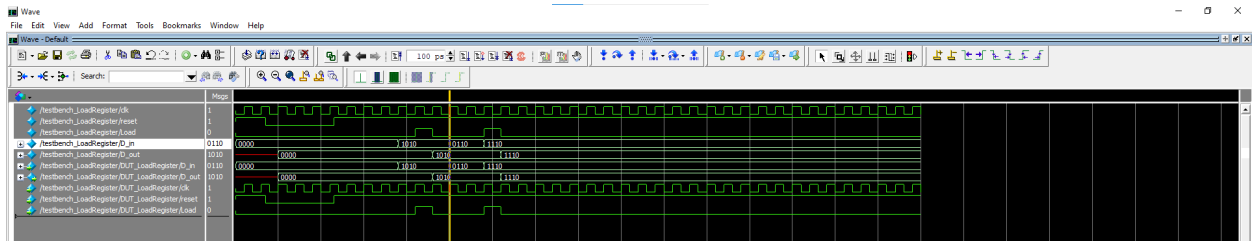
D_in is then loaded with the value of 4'b1010.



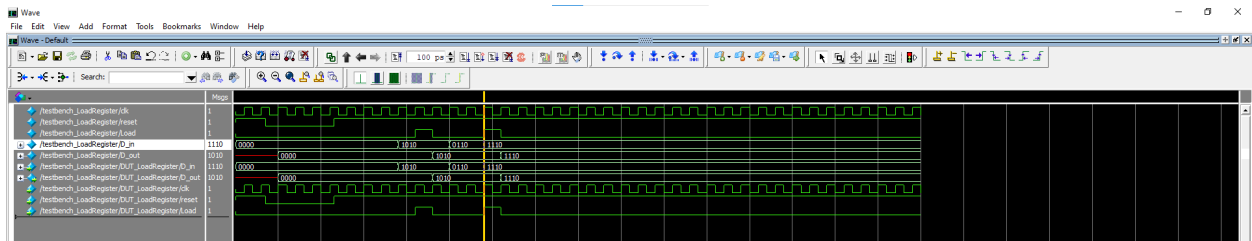
Load signal is high (loading enabled).



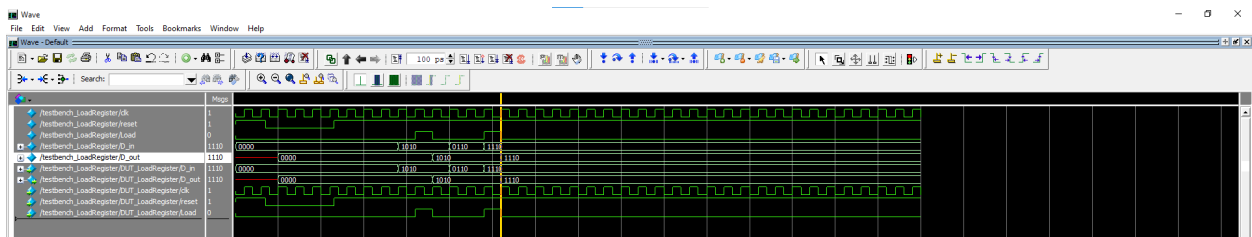
`D_out` is loaded with `D_in`'s value of 4'b1010 at the following rising edge of `clk` signal.



`D_in` is loaded with the value of 4'b0110. `D_out` isn't loaded with this value of `D_in` because `Load` signal isn't raised.

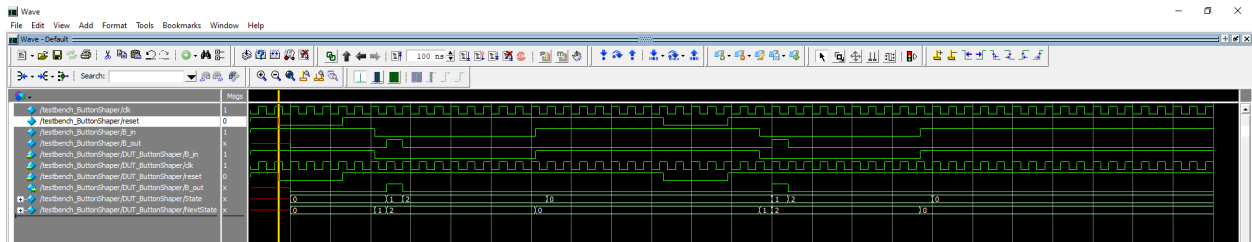


`D_in` is loaded with the value of 4'b1110 and the `Load` signal is raised at the same time.

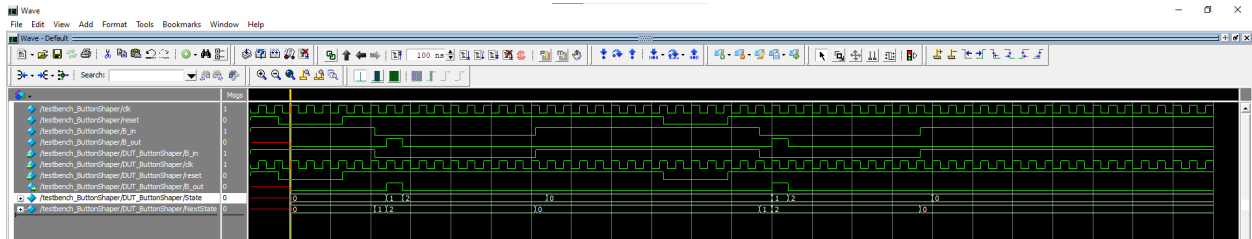


`D_out` is loaded with `D_in`'s value of 4'b1110 at the following rising edge of `clk` signal.

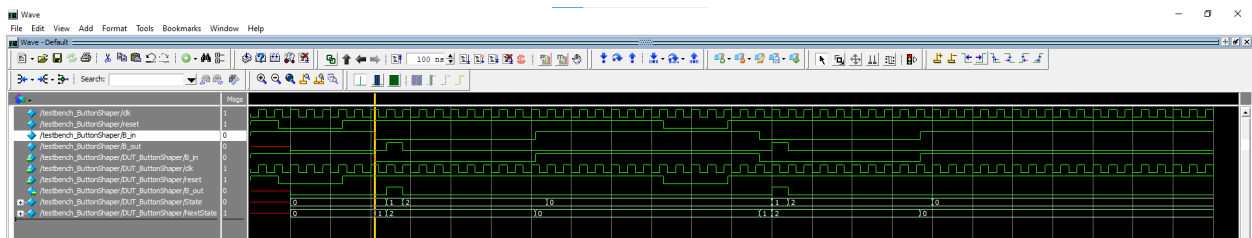
3.4. Button Shaper



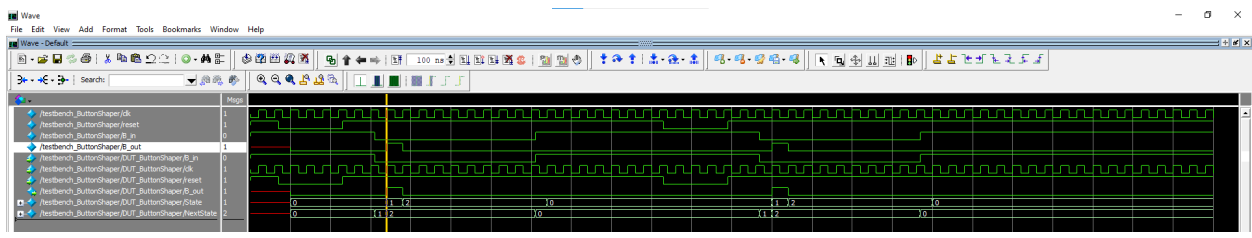
reset signal is pulled low/ active low (reset button is pushed).



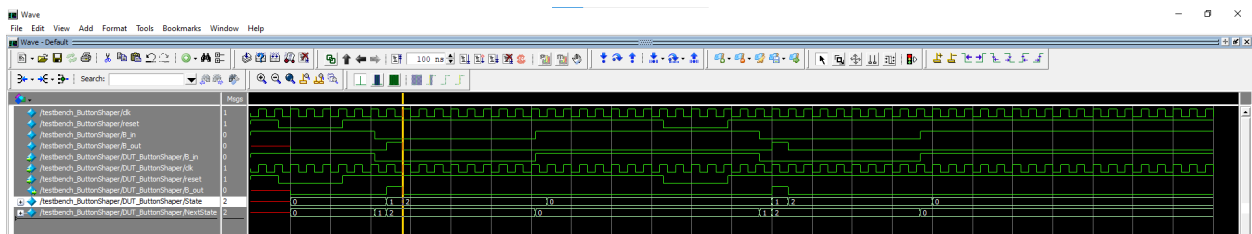
It gets into State 1 at the following rising edge of clk signal after reset signal is pulled low.



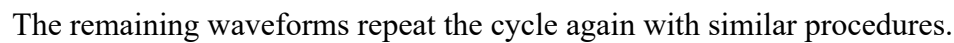
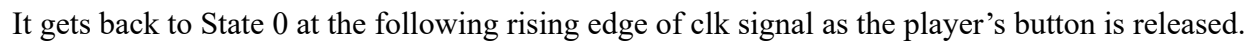
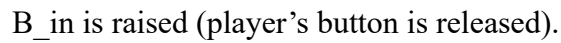
B_in is pulled low/ active low (a player's input button is pushed).



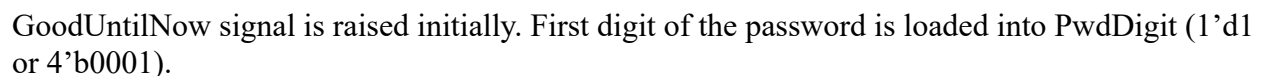
B_out is generated as an active high one-clock-cycle pulse signal at the rising edge of clk signal and it gets into State 2.

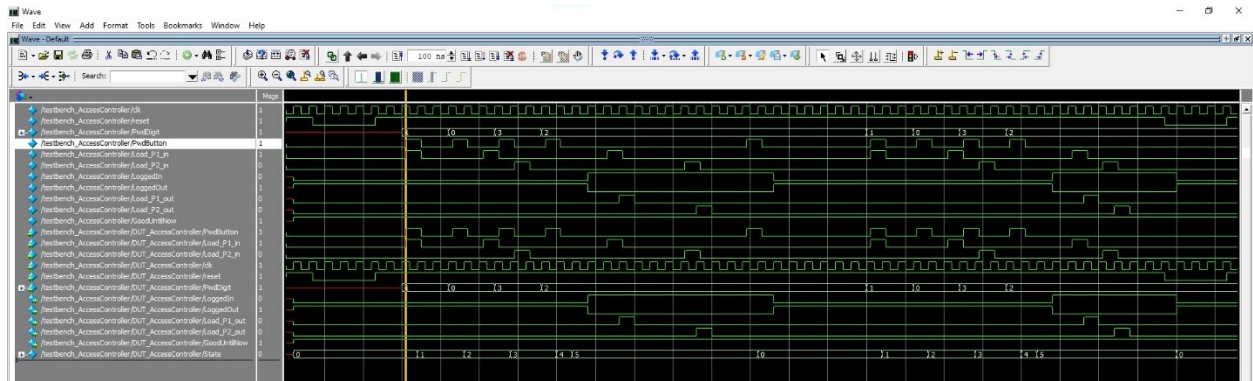


B_out is raised for exactly one clock cycle. At this point, it gets into State 2 waiting for the physical player's input button to be released.

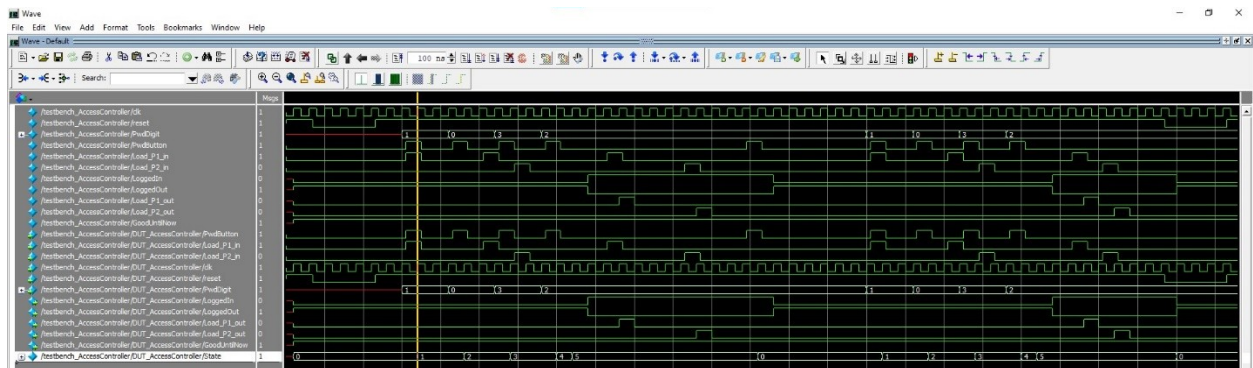


3.5.1. Correct password condition simulation

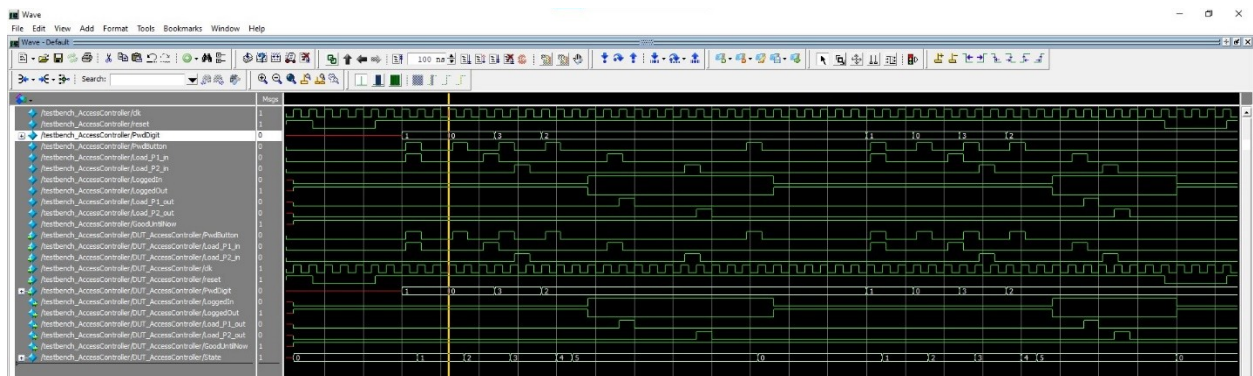




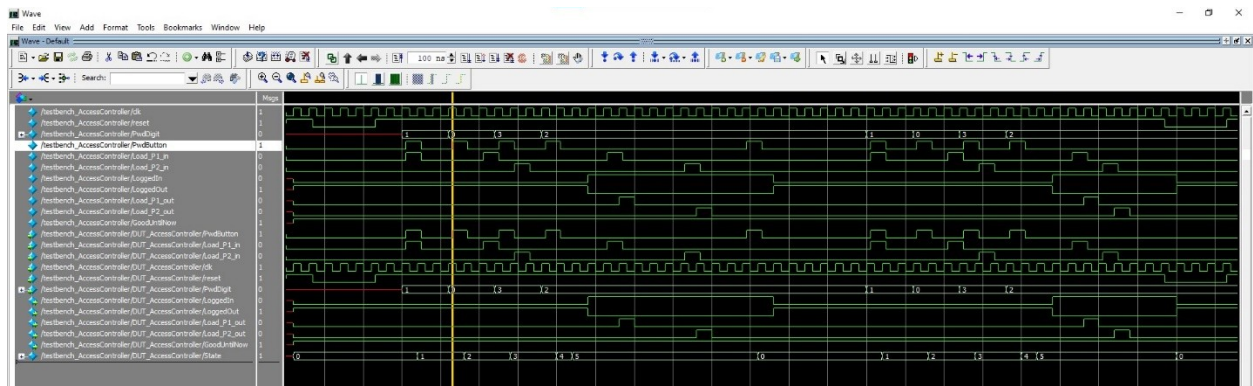
The first one-clock-cycle pulse signal PwdButton enables the loading of the first password digit. At the same time, Load_P1_in is raised indicating Player 1 pushes their LOAD button but Load_P1_out stays low. This means that players can't load their input numbers before entering the correct password.



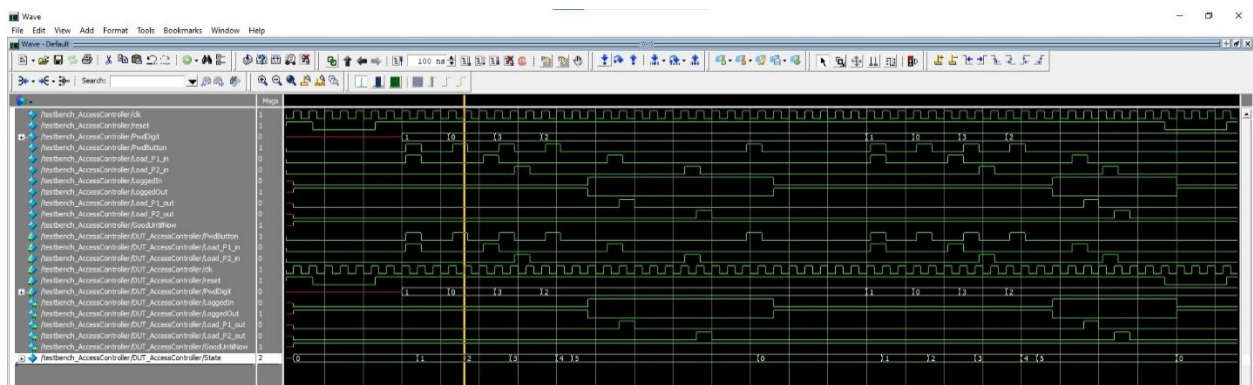
State changes from 0 to 1 at the next rising edge of clk signal. GoodUntilNow stays high meaning the first password digit is correct.



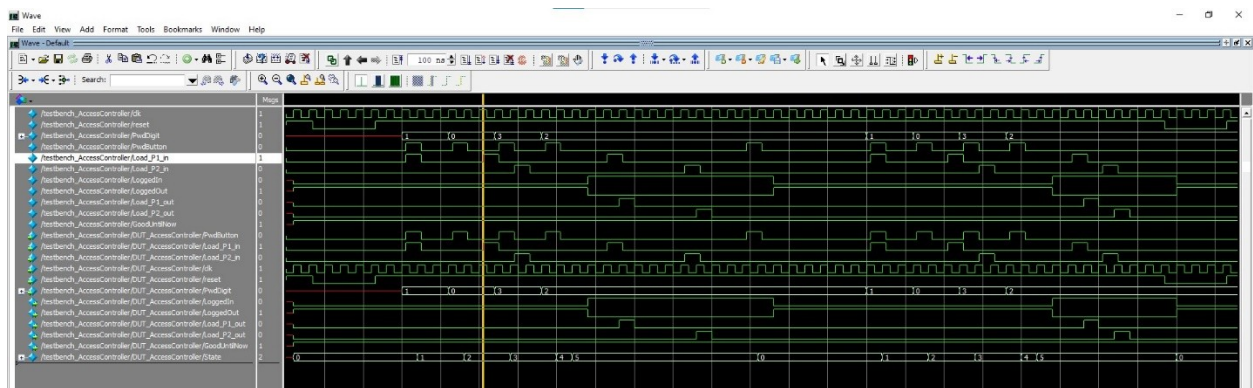
Second digit of the password is loaded into PwdDigit (1'd0 or 4'b0000).



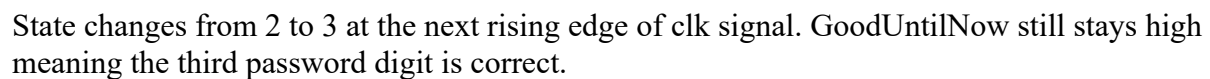
The second one-clock-cycle pulse signal PwddButton enables the loading of the second password digit.

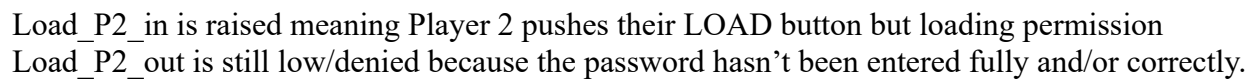


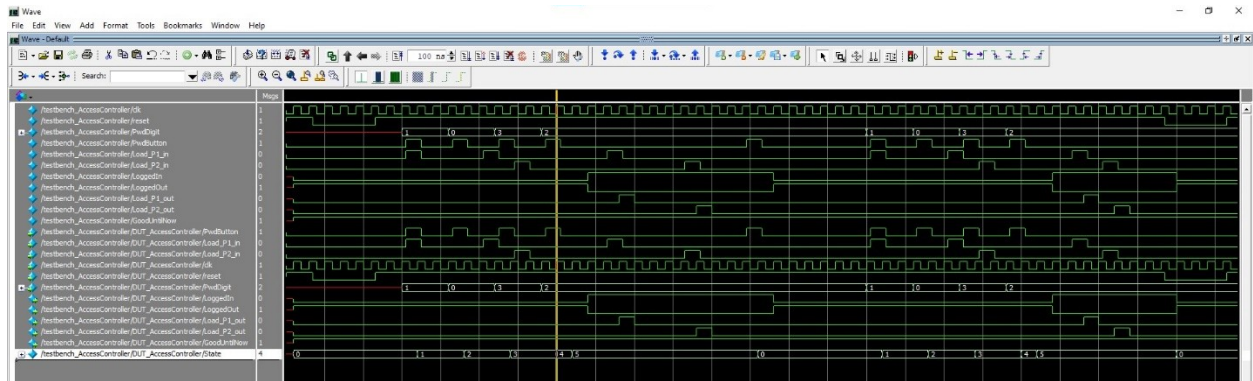
State changes from 1 to 2 at the next rising edge of clk signal. GoodUntilNow still stays high meaning the second password digit is correct.



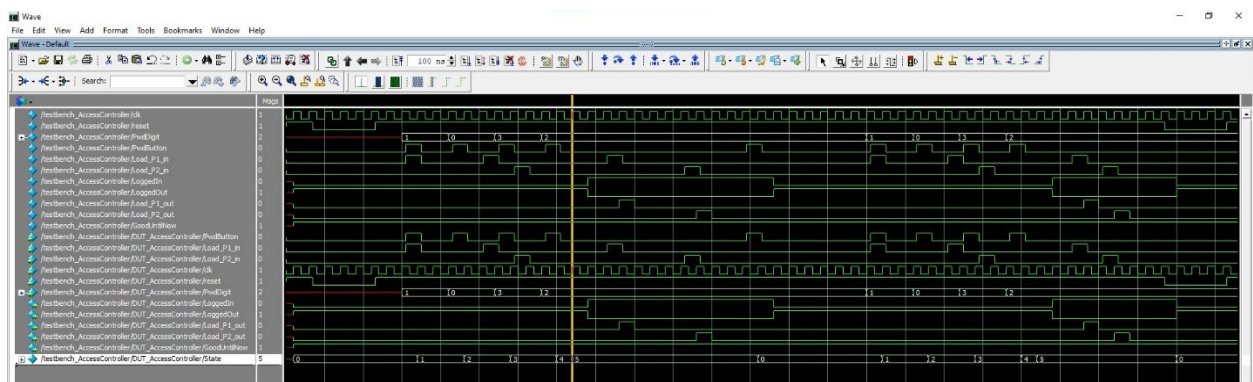
Load_P1_in is raised again meaning Player 1 pushes their LOAD button but loading permission Load_P1_out is still low/denied because the password hasn't been entered fully and/or correctly.



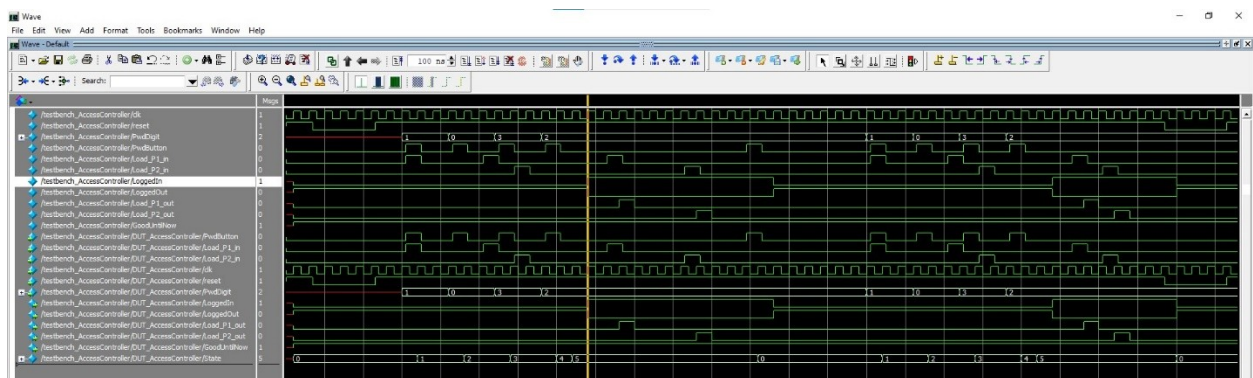




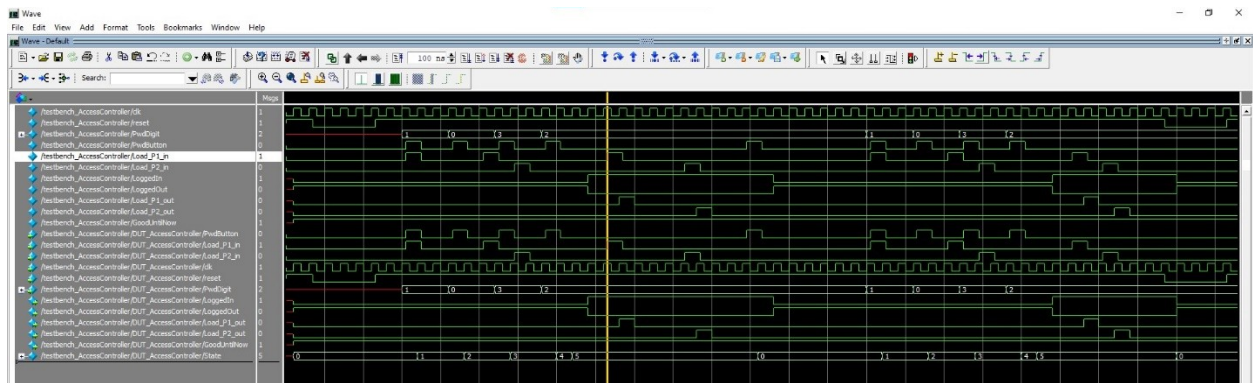
State changes from 3 to 4 at the next rising edge of clk signal. GoodUntilNow still stays high meaning all four password digits are correctly entered.



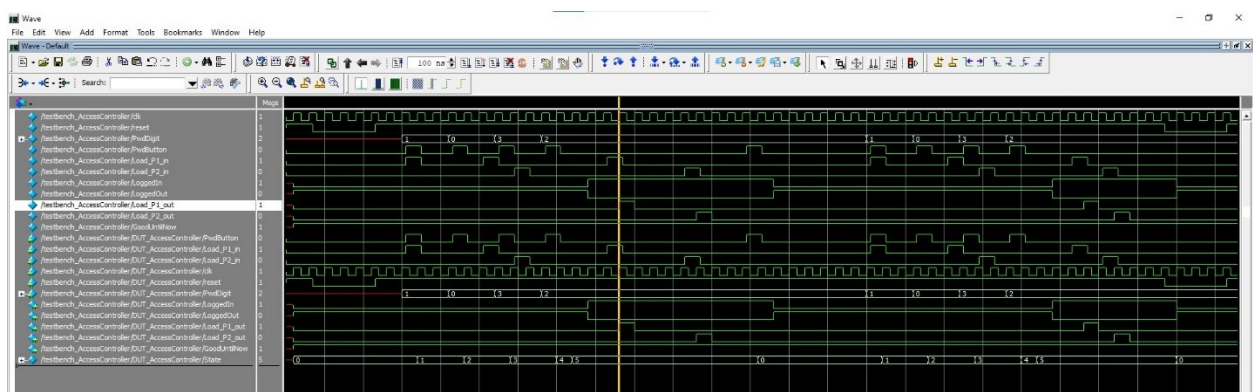
State changes from 4 to 5 at the next rising edge of clk signal.



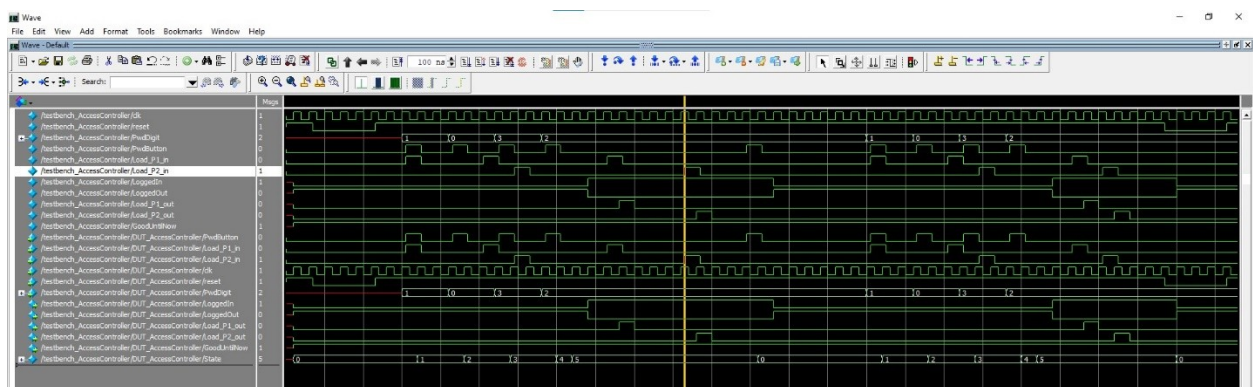
LoggedIn is raised at the next rising edge of clk signal and LoggedOut is pulled low. This indicates that the password is correct and players are logged in.



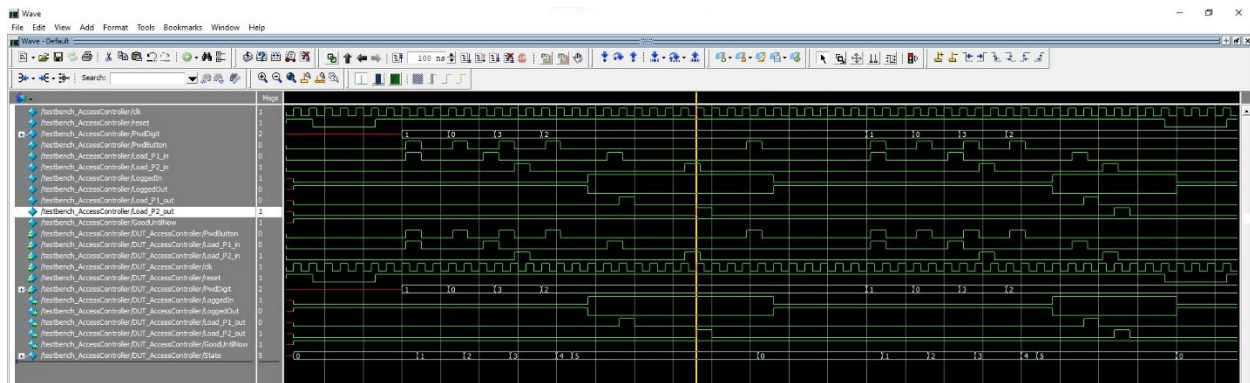
Player 1 now pushes their LOAD button again as Load_P1_in is raised.



Load_P1_out is raised at the next rising edge of clk signal. This means the players can load their numbers now as they're logged in.

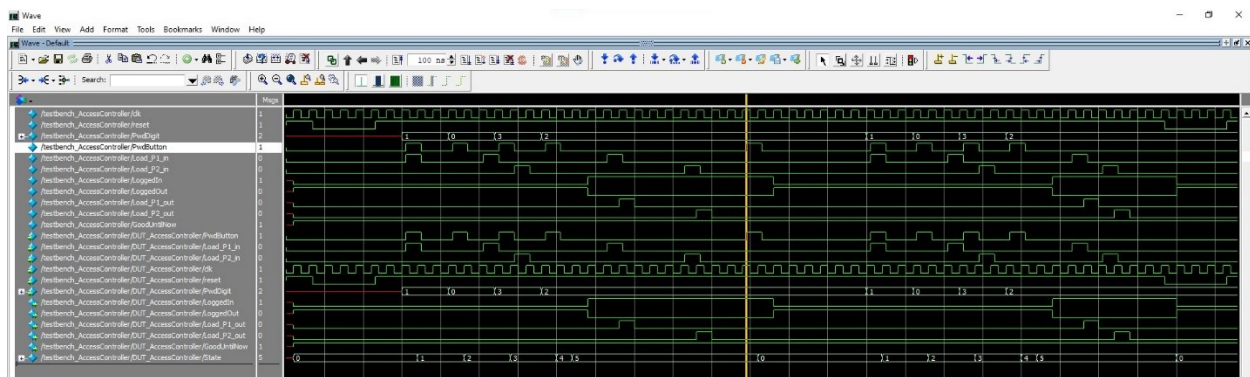


Player 2 pushes their LOAD button as Load_P2_in is raised.

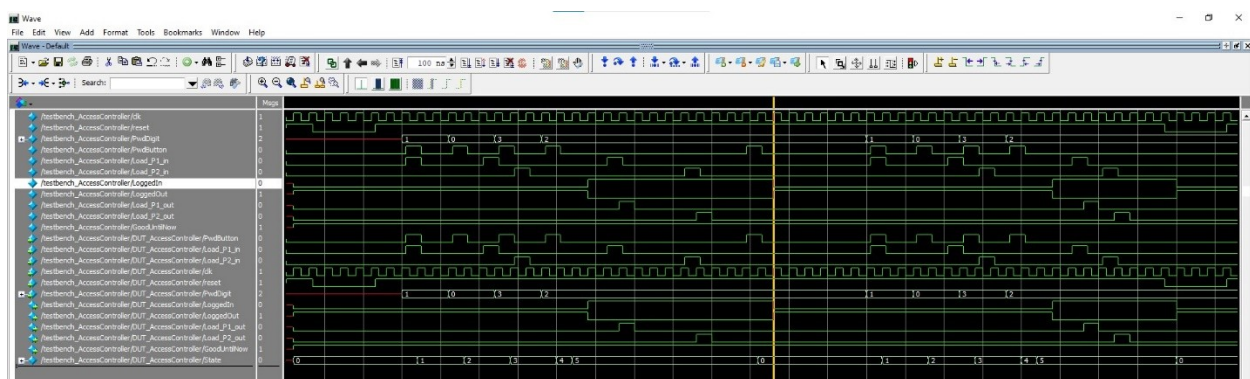


Load_P2_out is raised at the next rising edge of clk signal.

Bonus feature:



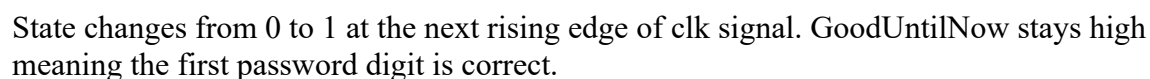
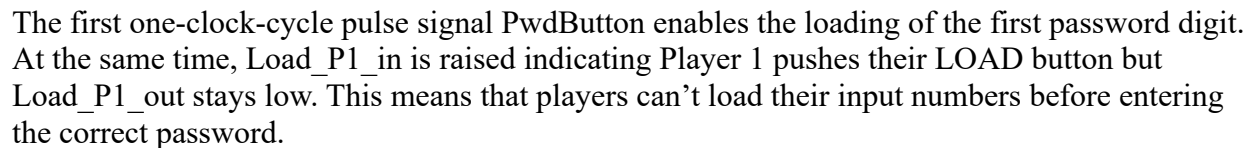
The password load button is pushed when players are logged in as PwdButton is raised.

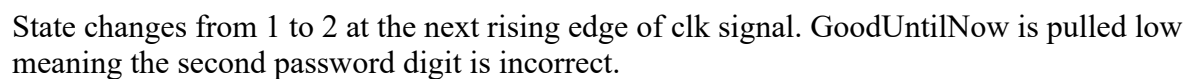
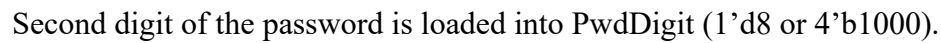


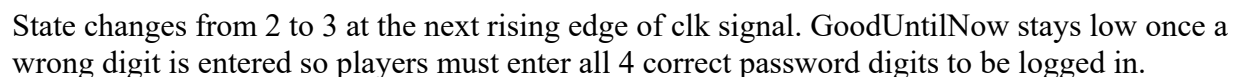
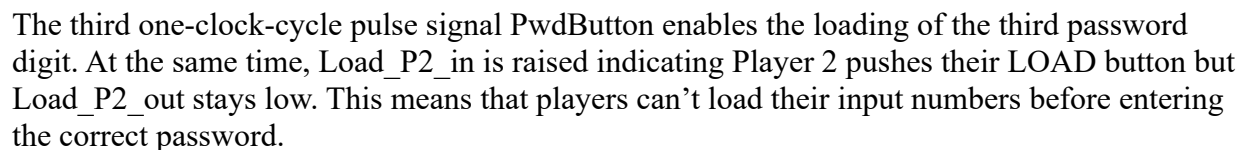
LoggedIn is pulled low and LoggedOut is raised at the next rising edge of clk signal. This means players are logged out again after the previous pass load button push.

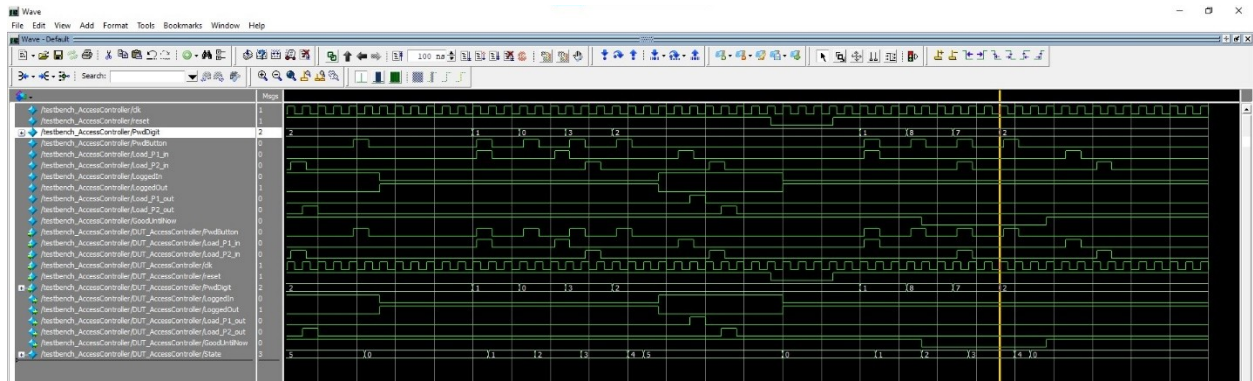
The next waveforms indicate that players try to log in again with the same password (1032).

3.5.1. Wrong password condition simulation

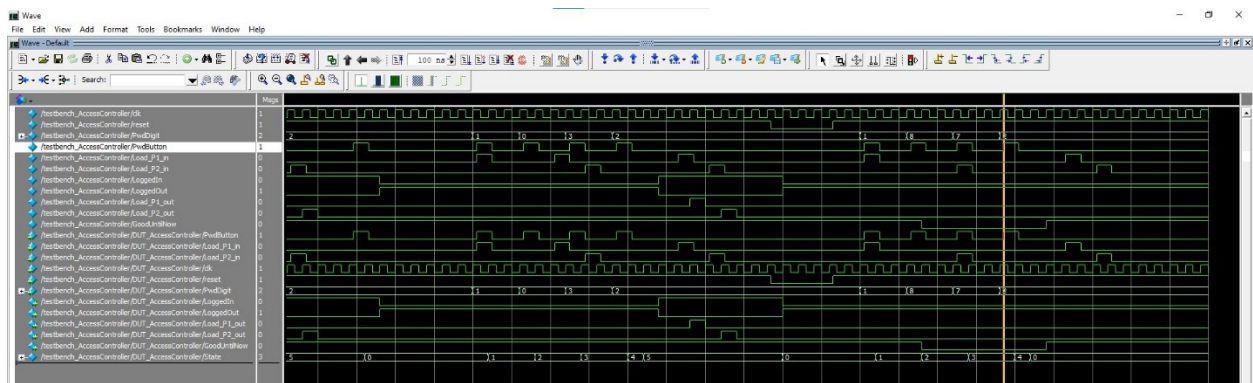




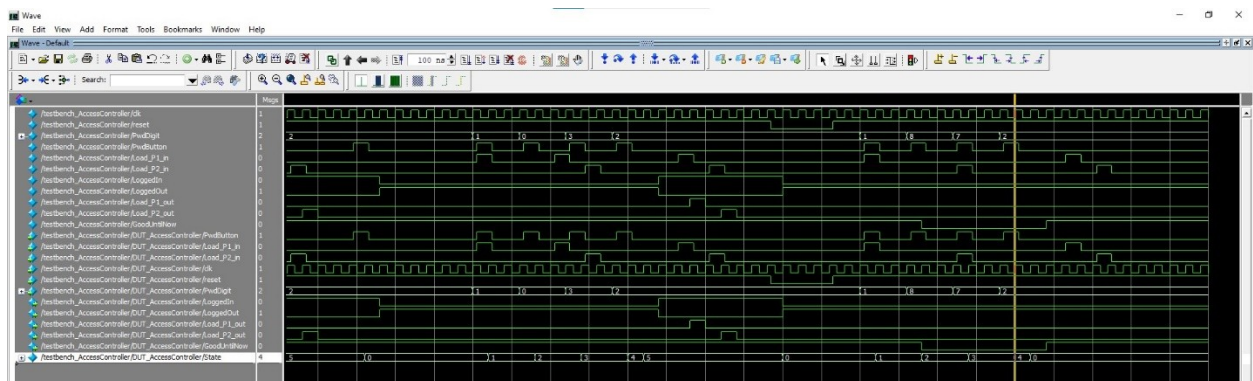




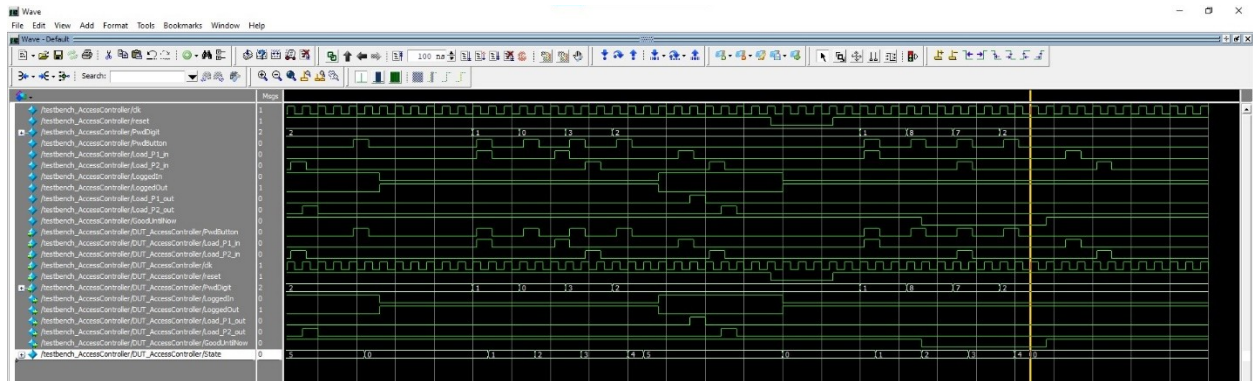
Last digit of the password is loaded into PwdDigit (1'd2 or 4'b0010).



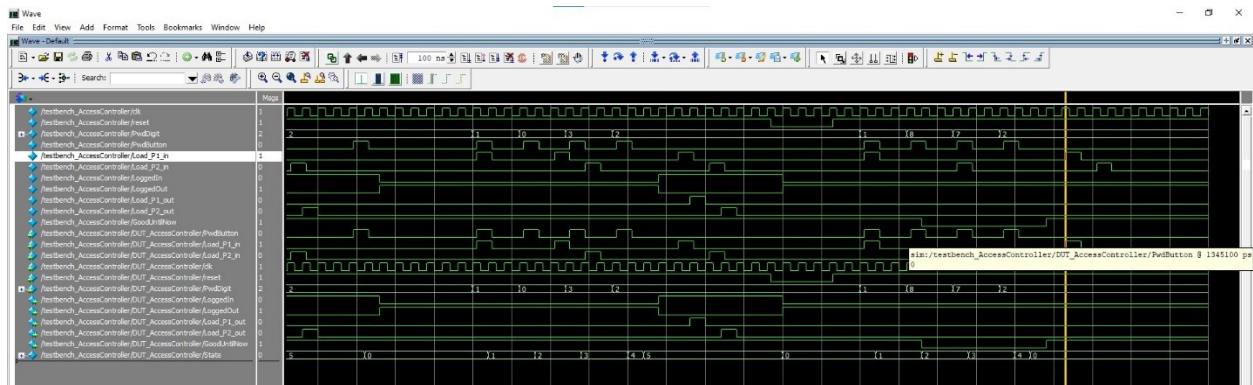
The last one-clock-cycle pulse signal PwdButton enables the loading of the last password digit.



State changes from 3 to 4 at the next rising edge of clk signal. GoodUntilNow still stays low even the last password digit is correctly entered.



State changes from 4 to 0 at the next rising edge of clk signal. This means the password is incorrectly entered. Players must enter the password again. Through out this process, LoggedOut stays high while LoggedIn stays low the entire time.

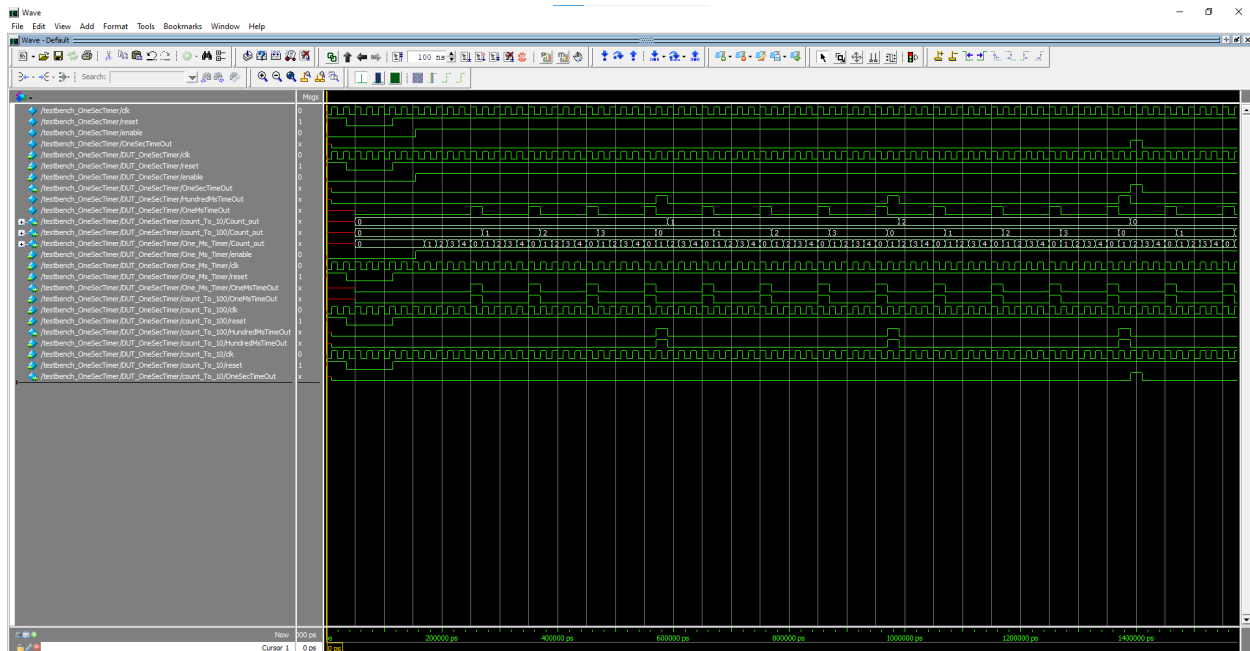


Player 1 pushes their LOAD button as Load_P1_in is raised. As can be seen at the next rising edge of clk signal, Load_P1_out stays low because players are not logged in to load their numbers.

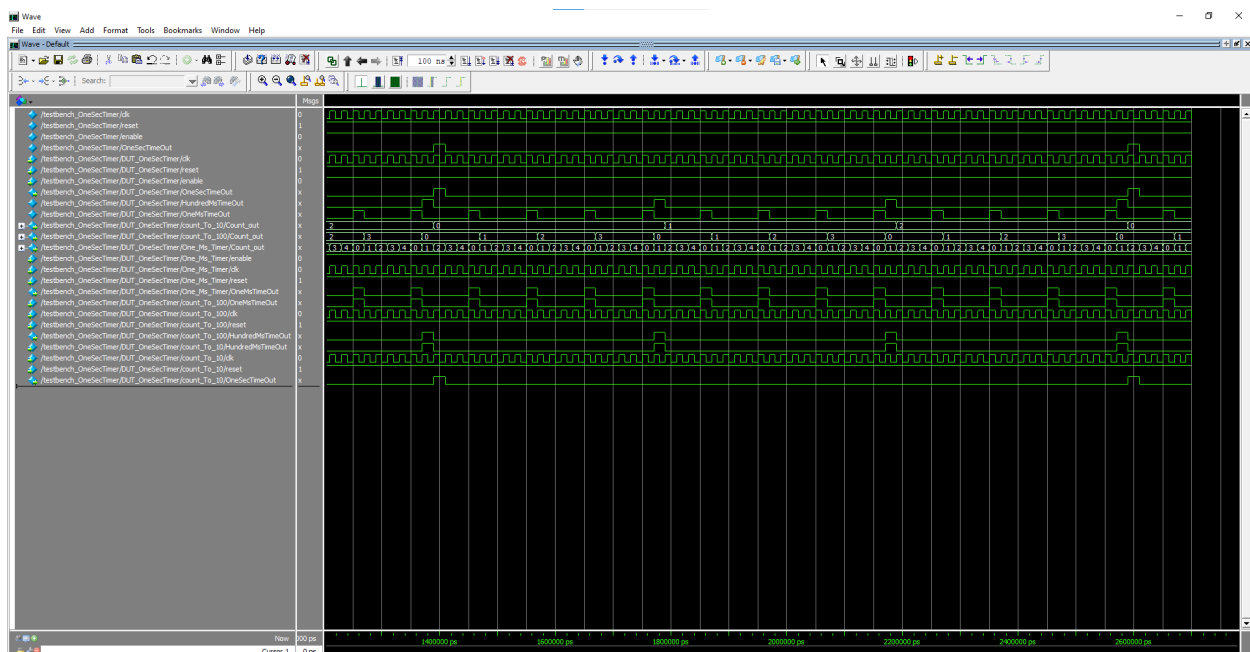


The same case for Player 2 LOAD button push.

3.6. One Second Timer



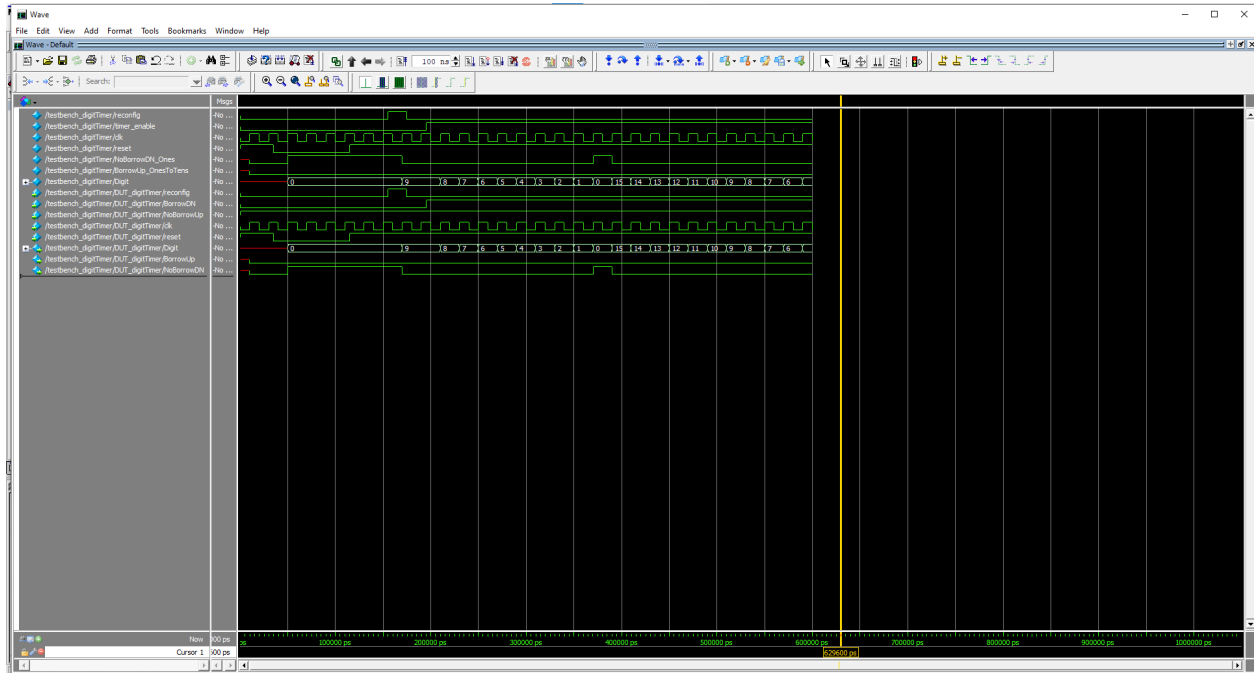
Simulation waveform showing the first pulse of OneSecTimeout signal



Simulation waveform showing the second pulse of OneSecTimeout signal

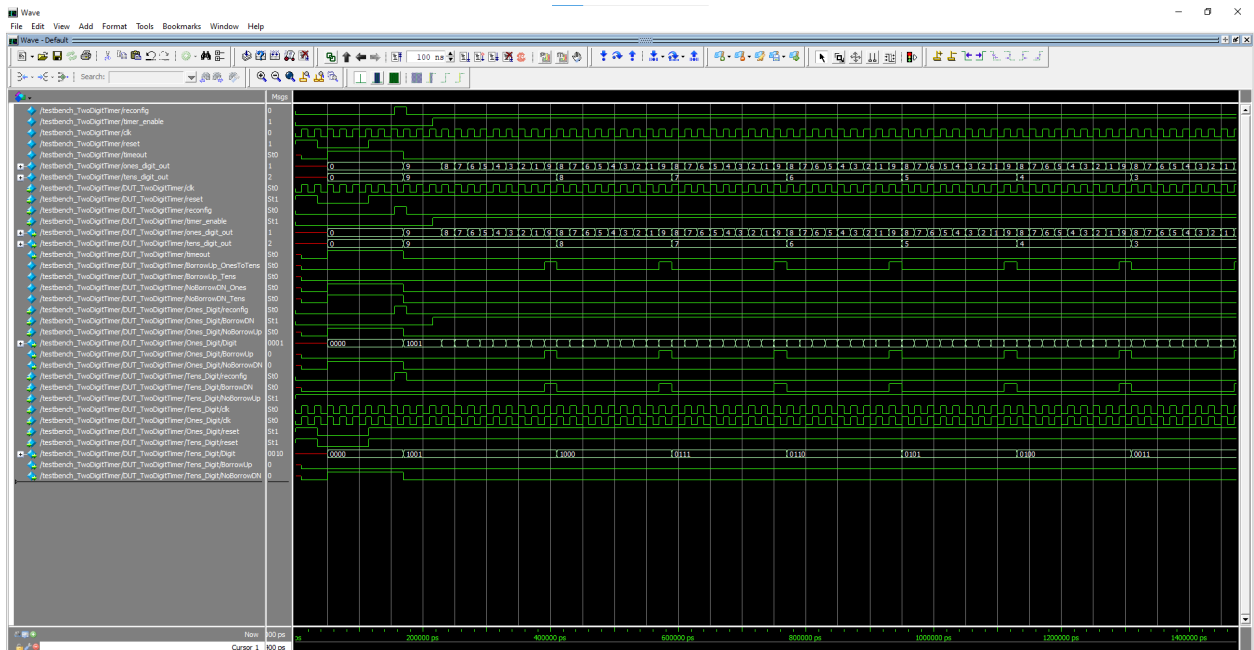
Simulation results show 2 one-clock-cycle pulses of the OneSecTimeout signal. These pulses occur once for every 3 one-clock-cycle pulses of the HundredMsTimeout and every 12 one-clock-cycle pulses of the OneMsTimeout (*Note that this is for simulation purposes only*).

3.7. 1 Digit Timer

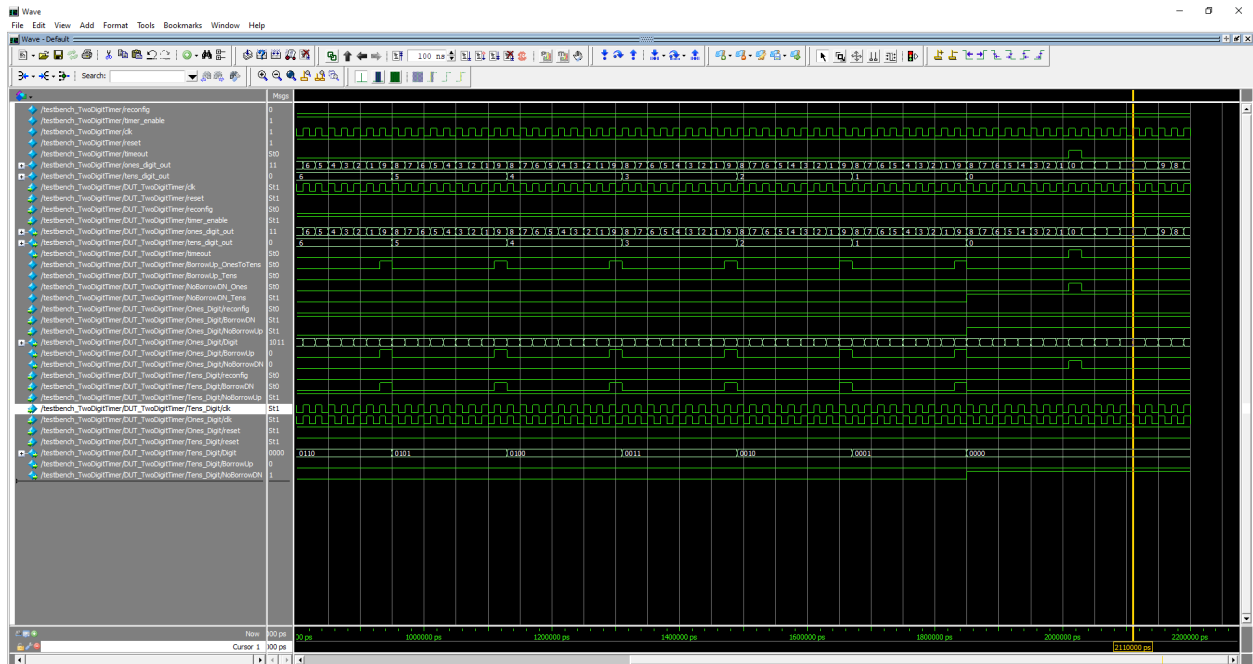


Digit timer simulation

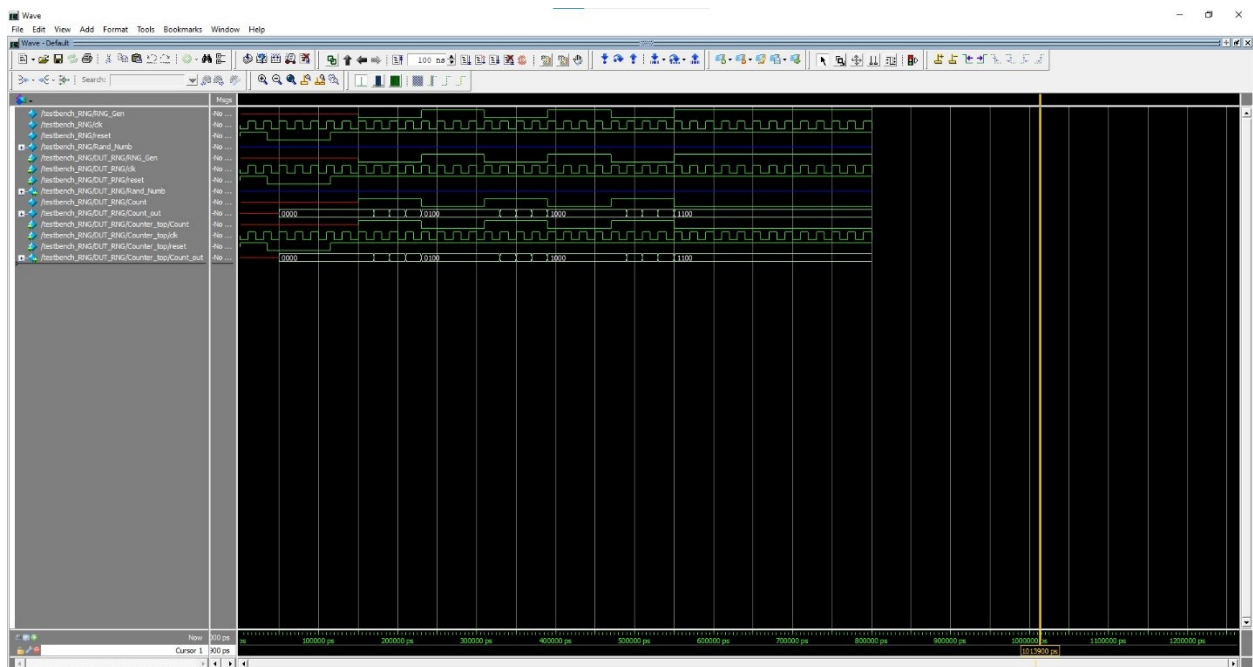
3.7.2 Two Digit Timer



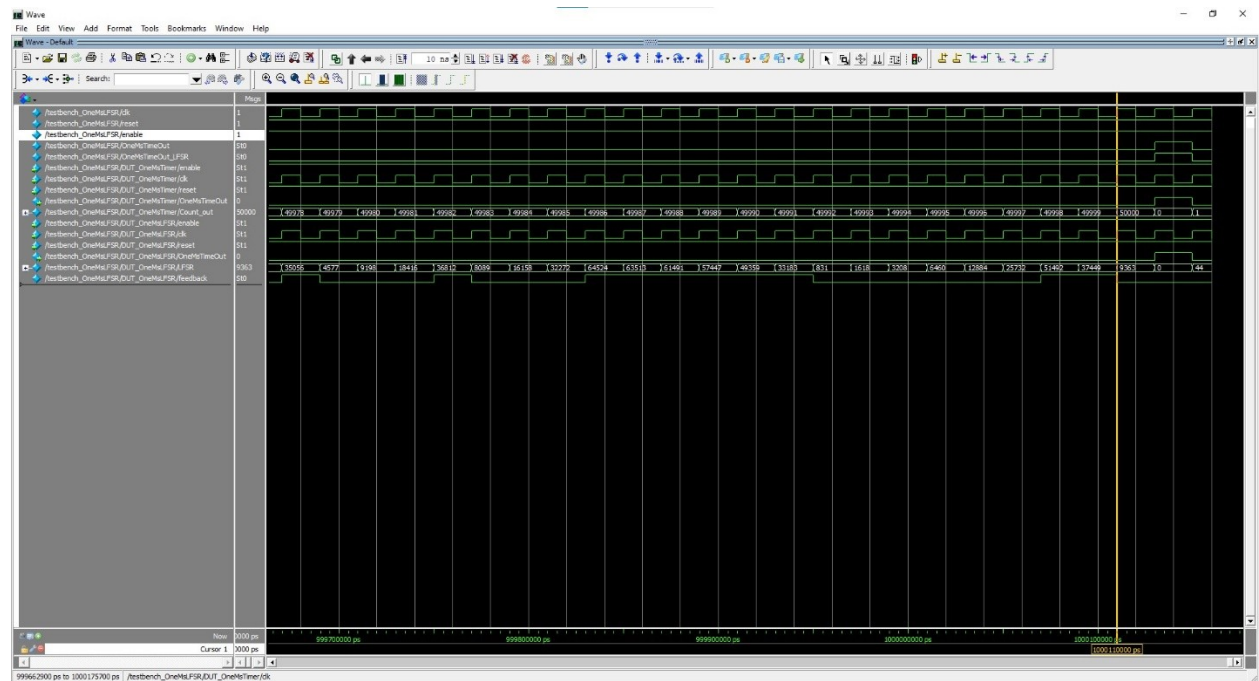
Two digit timer simulation



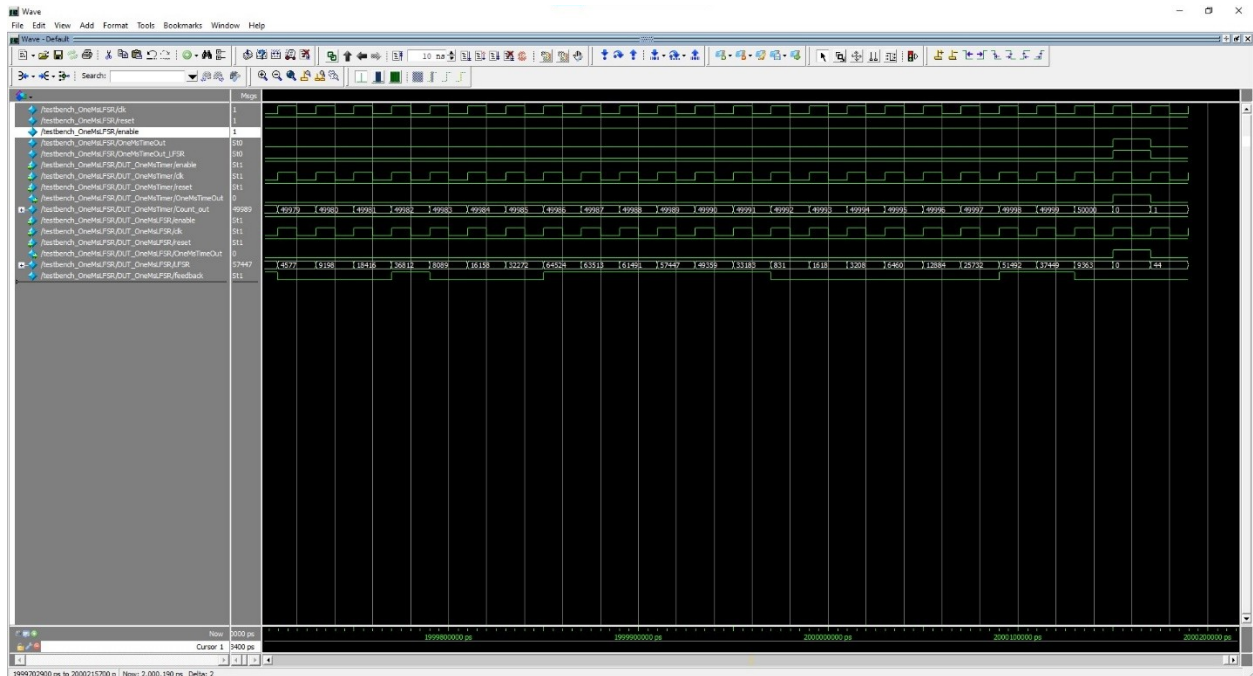
3.8. Random Number Generator



Internal count value of the LFSR counter is 16'd9363 equivalently to 16'd50000 of the original counter within OneMsTimer.

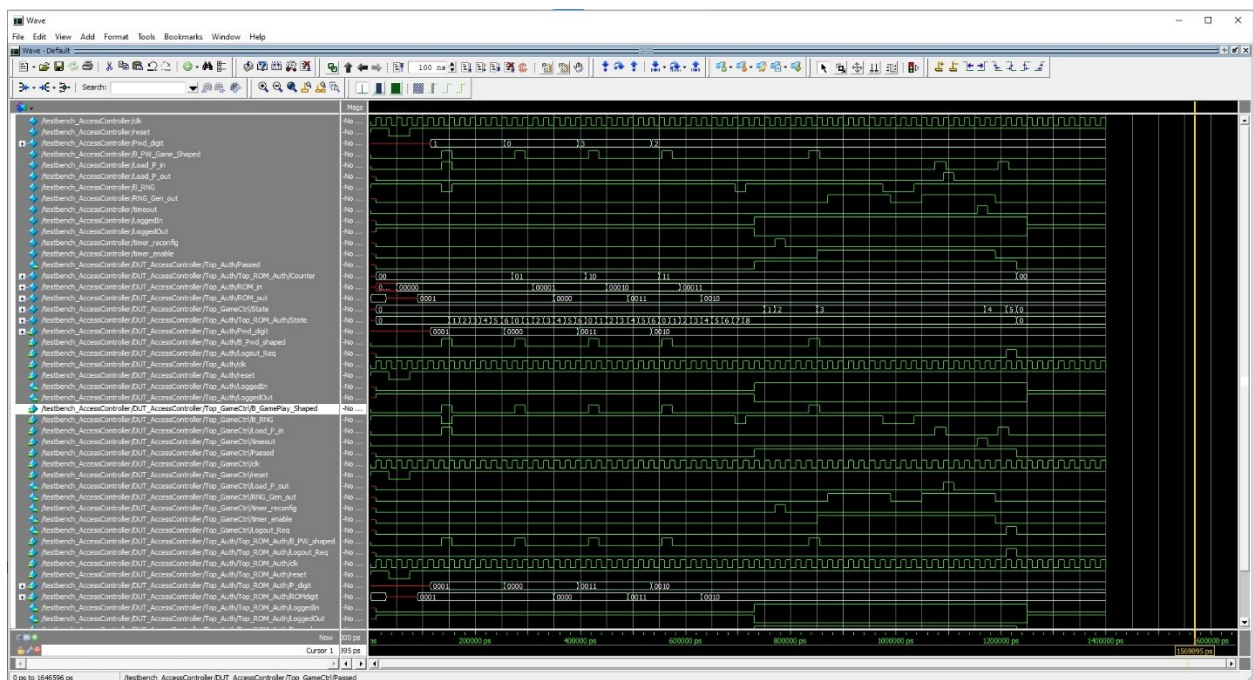


First 1 [ms] timeout pulse for side-by-side LFSR and original OneMsTimer counter



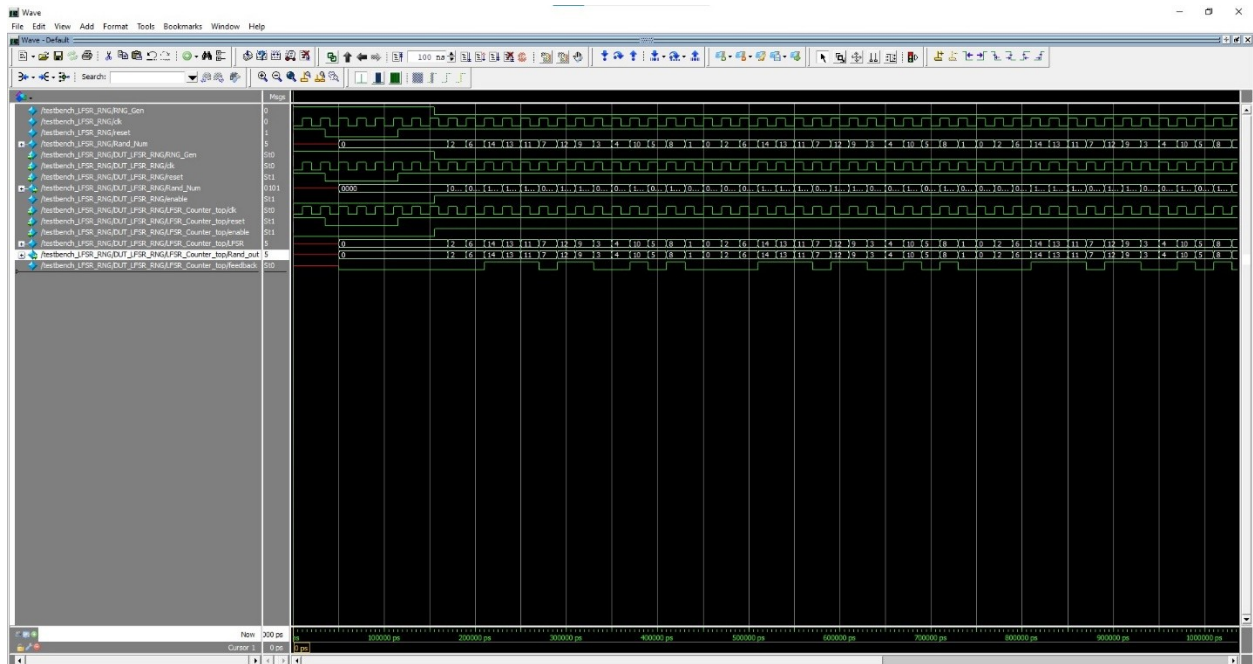
Second 1 [ms] timeout pulse for side-by-side LFSR and original OneMsTimer counter

3.10. Access Controller with separate Authentication and Gameplay Control



Access_top (access controller) simulation with integrated Top_Auth (password authentication) and Top_GameCtrl (gameplay control)

3.11. LFSR-based Random Number Generator



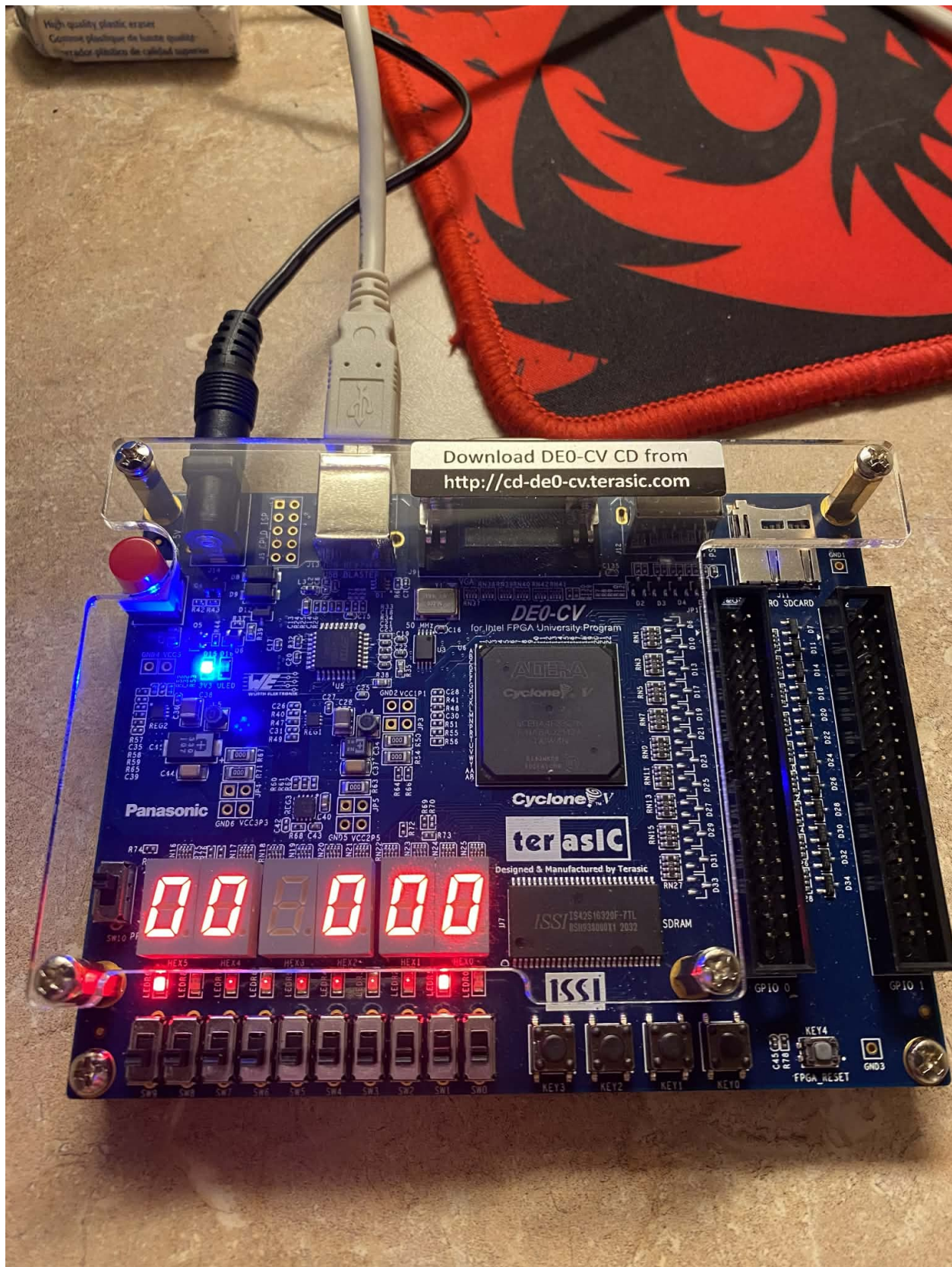


Figure 5: FPGA default settings

Step 2: After correctly entering all four password digits, timer will display 99 and rightmost LED is lit meaning it's logged in.

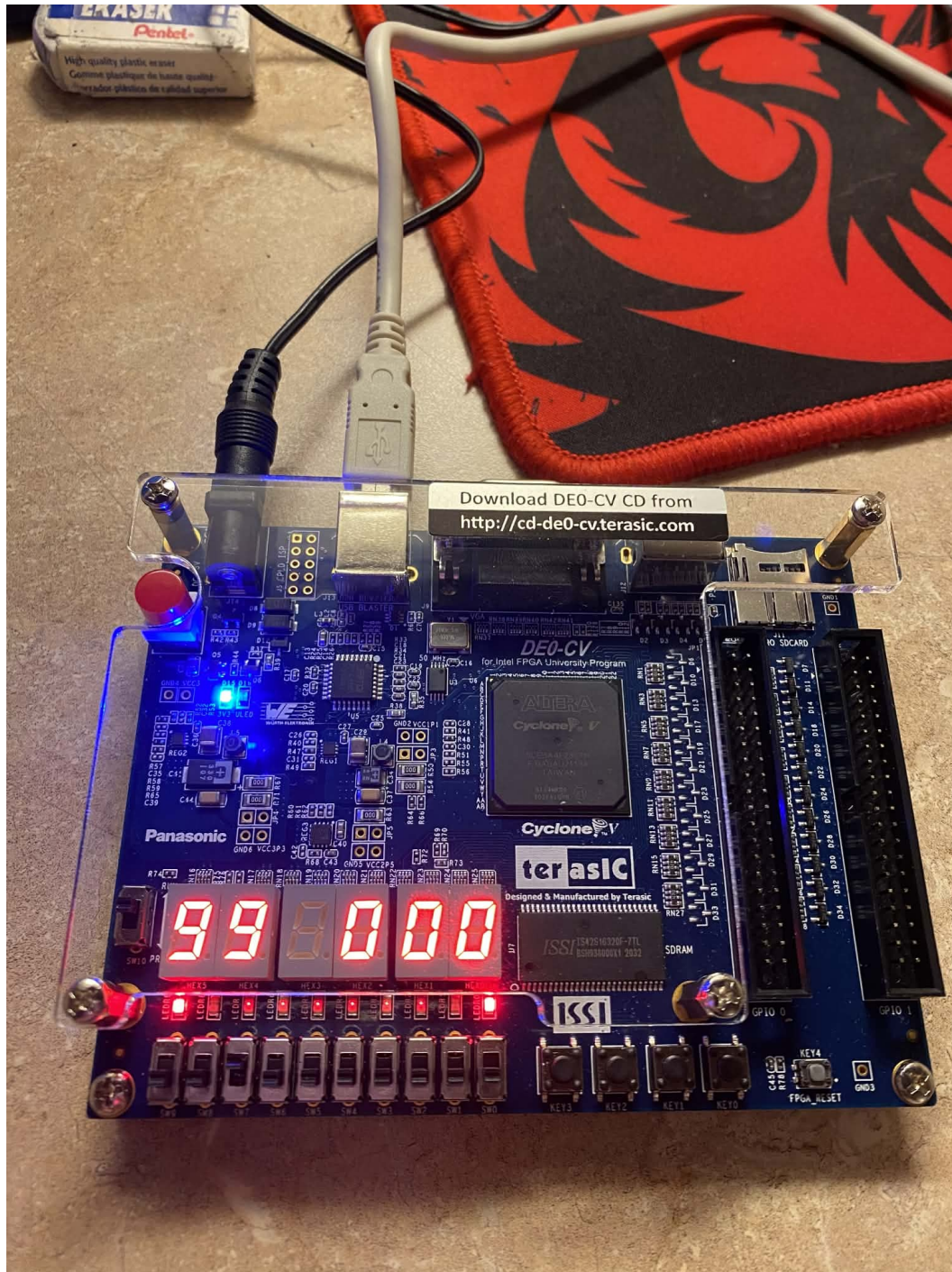


Figure 6: Status after successfully logged in

Step 3: Pressing the Game button will have the timer start counting down.

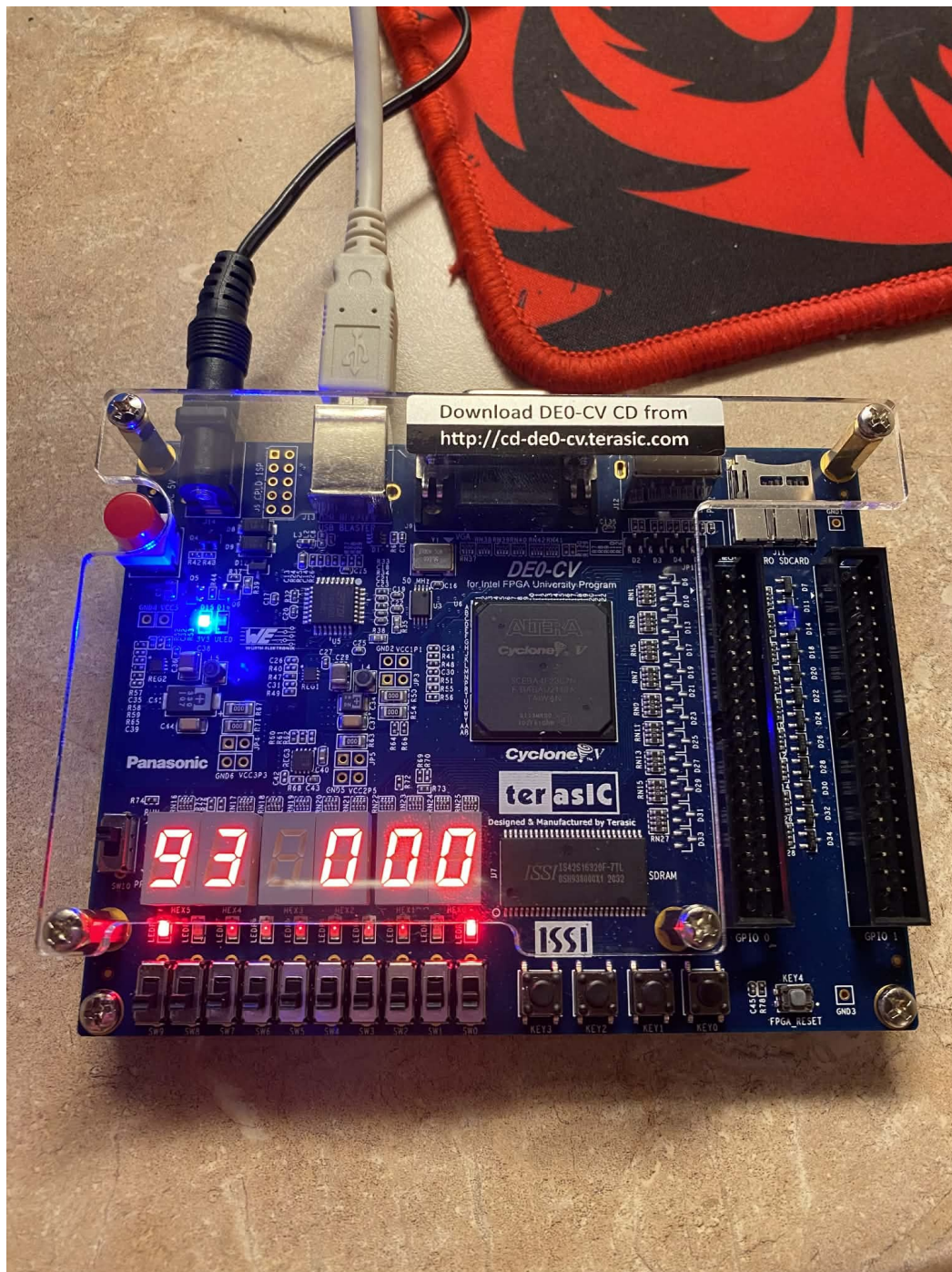


Figure 7: Timer starts counting

Step 4: Pressing the random number generation button will display a random number on the next-to-rightmost display for Player to match.

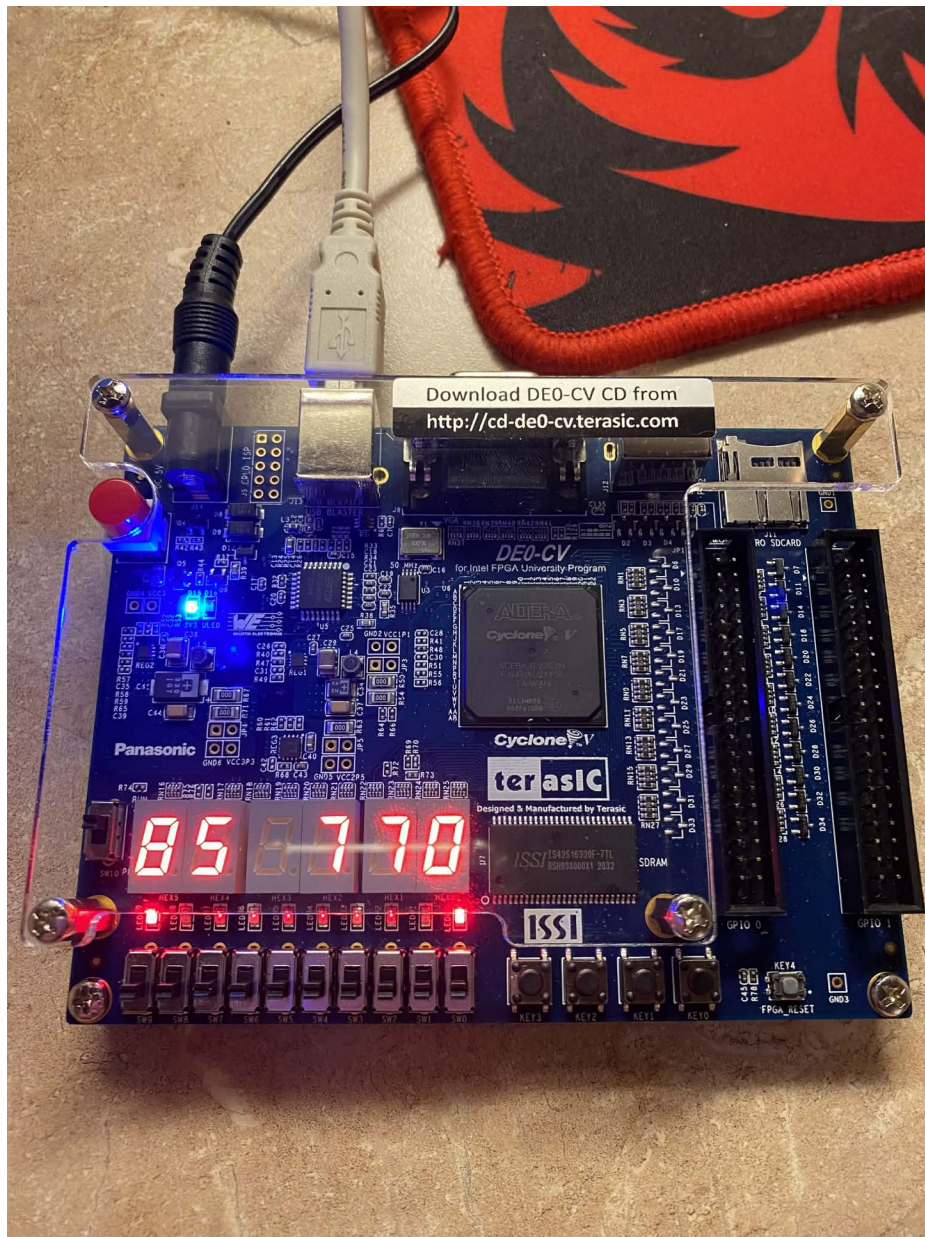


Figure 8: Random number is displayed

Step 5: Player enters a matching number (8) and pushes the LOAD button to load their number. The third-from-rightmost display shows the sum of two numbers (F means a matching pair is found). Next-to-leftmost LED is lit indicating matching condition.

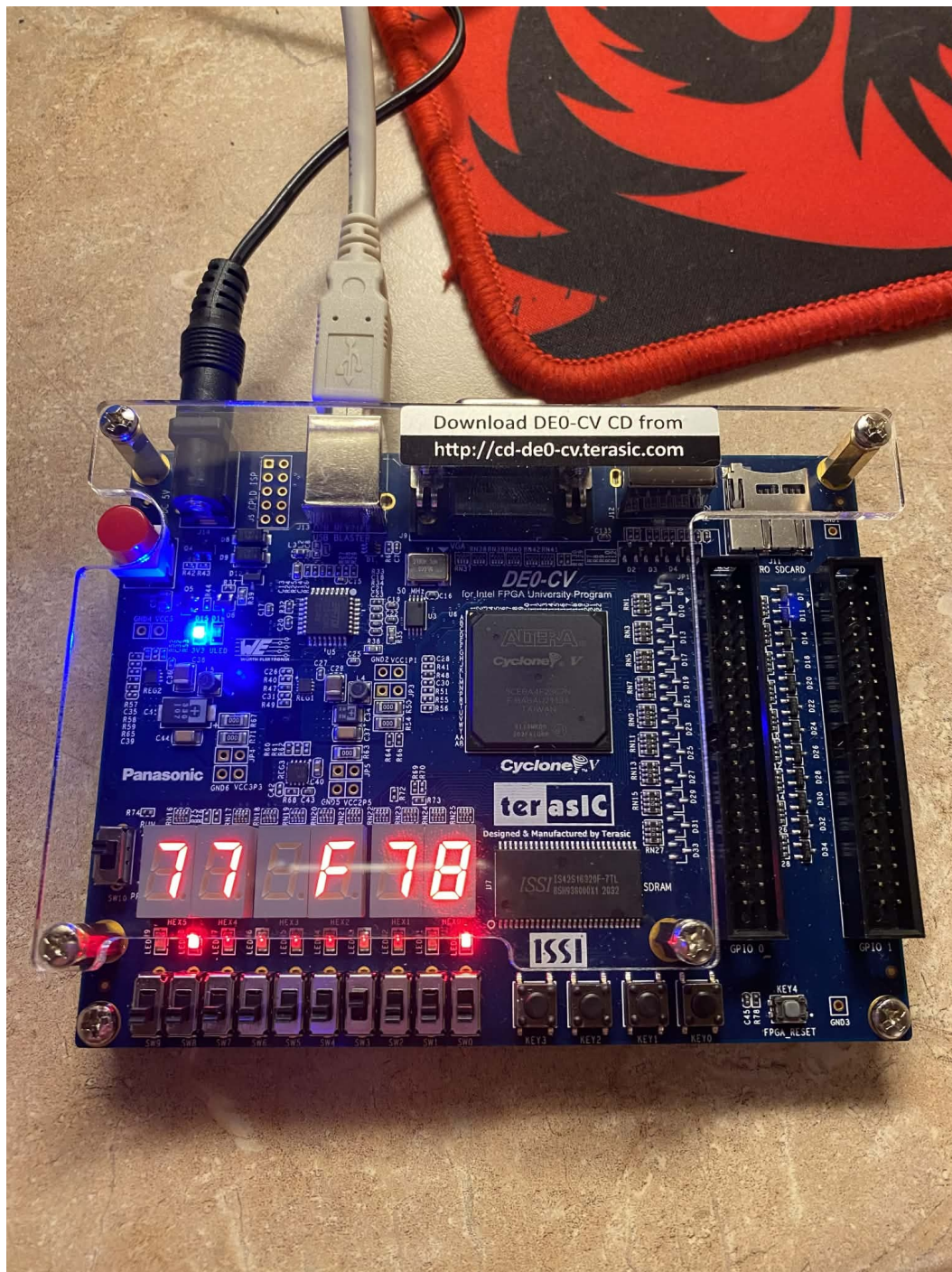


Figure 9: Matching number is found and LED is lit

Another matching pair is found. Timer still ticking down.

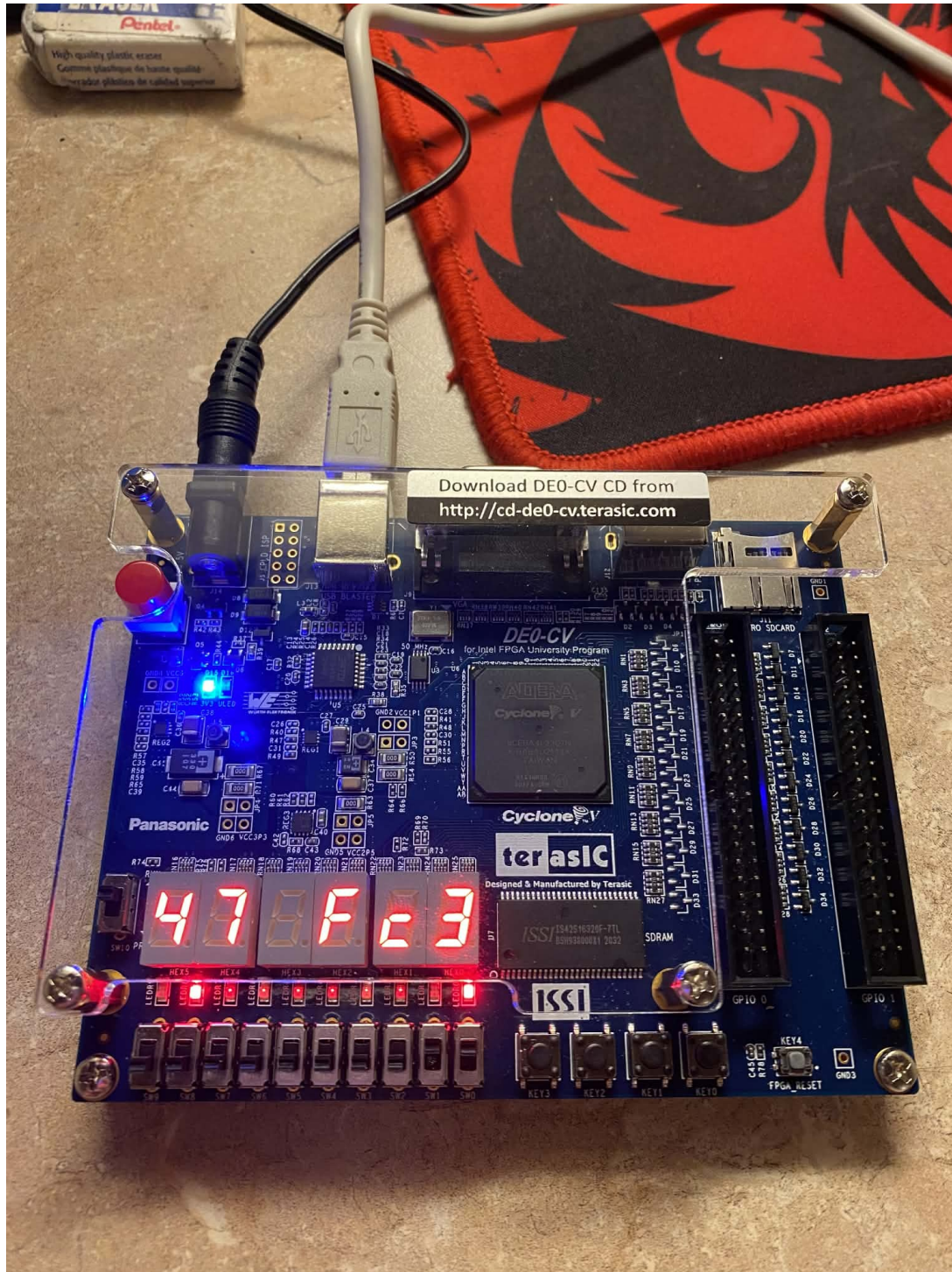


Figure 10: Another matching pair is found

Step 7: When timer hits 00, gameplay is disabled.

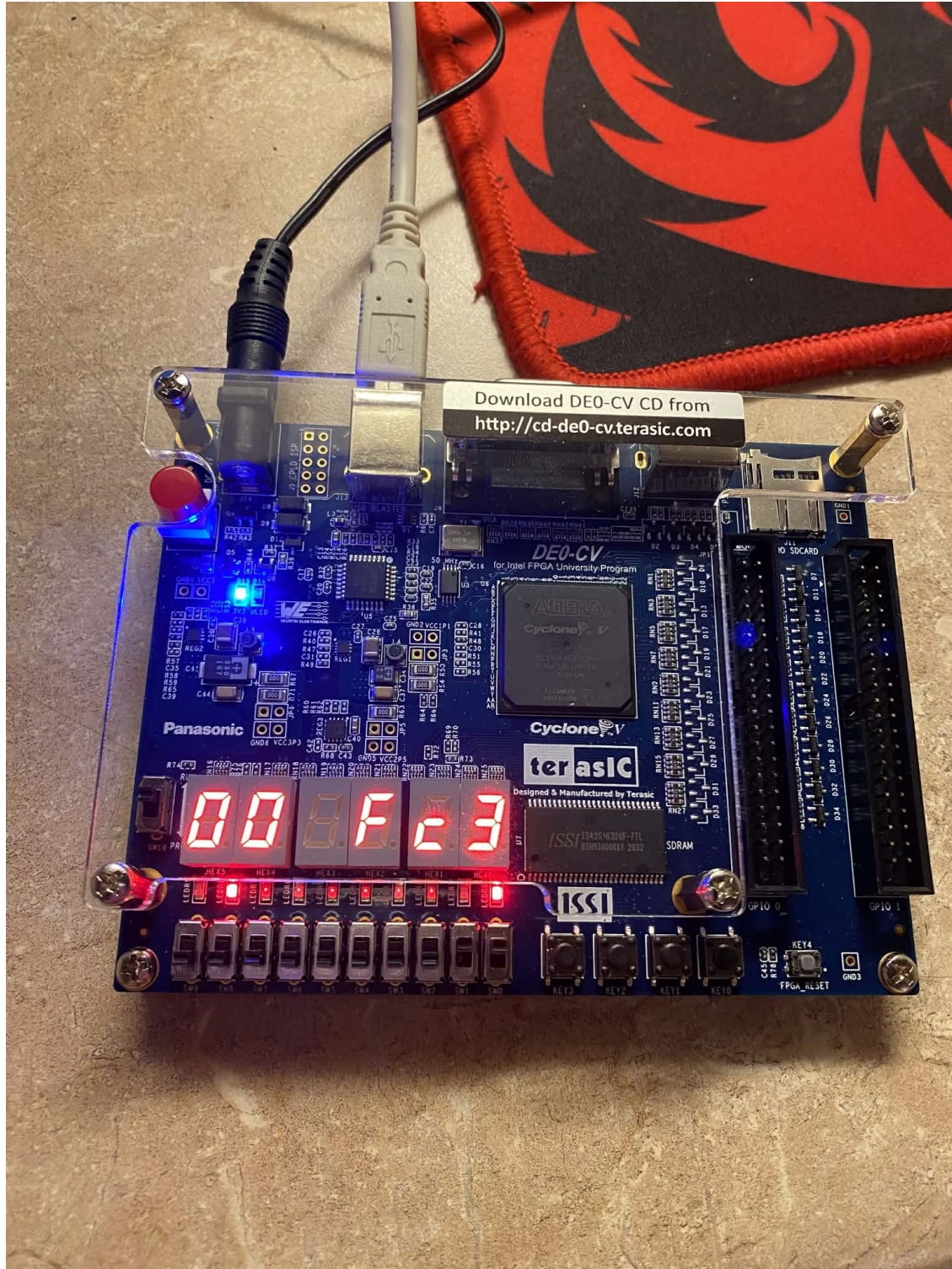


Figure 11: Game over status

Step 8: Pressing the Game/GameRestart button again will restart the timer. Pressing that same button again will start the timer and Player can start playing again.

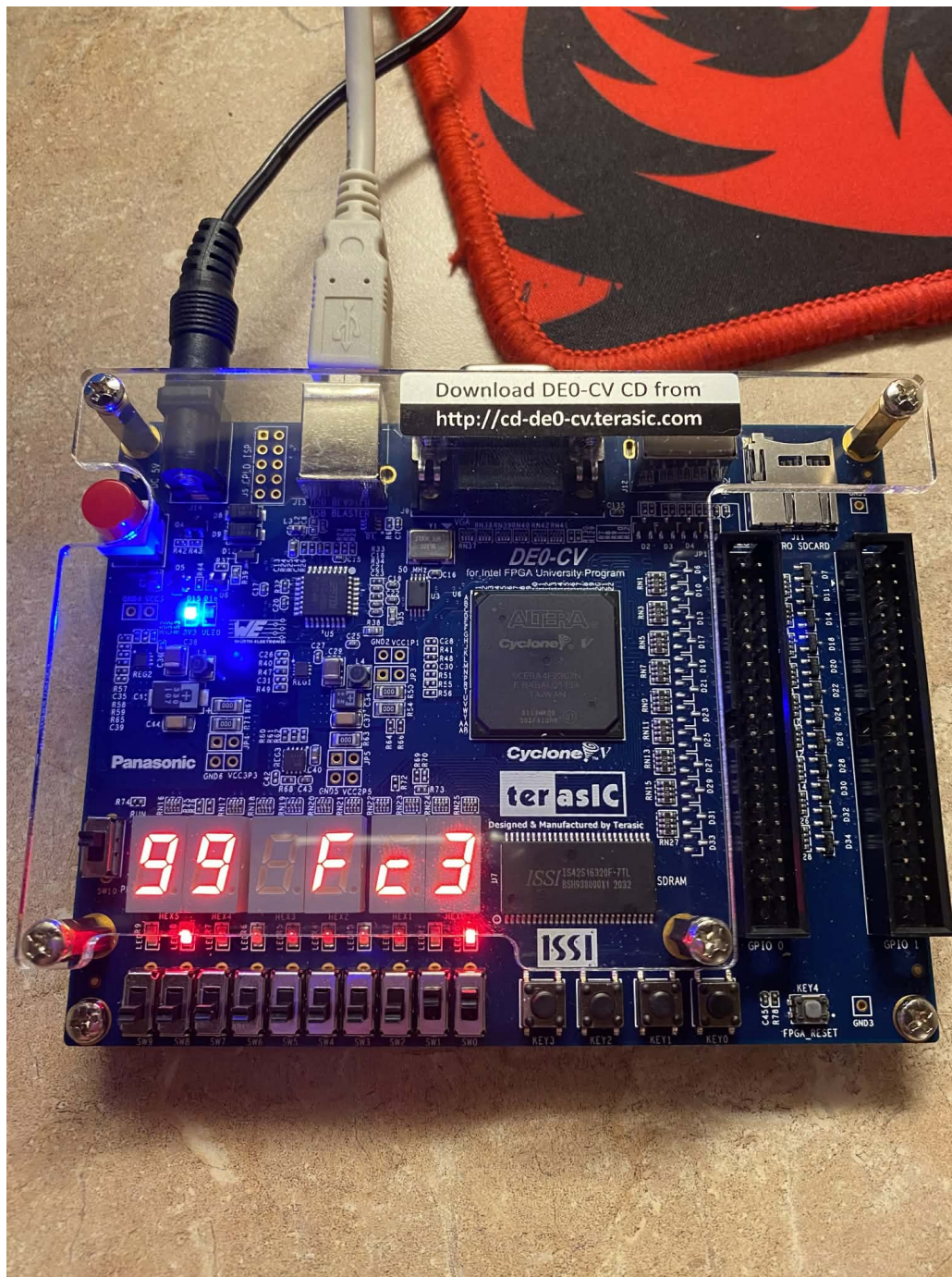


Figure 12: Timer is reloaded and game is restarted

Step 9 (log out function): Pressing the Player LOAD button outside of gameplay state will trigger a log out request.

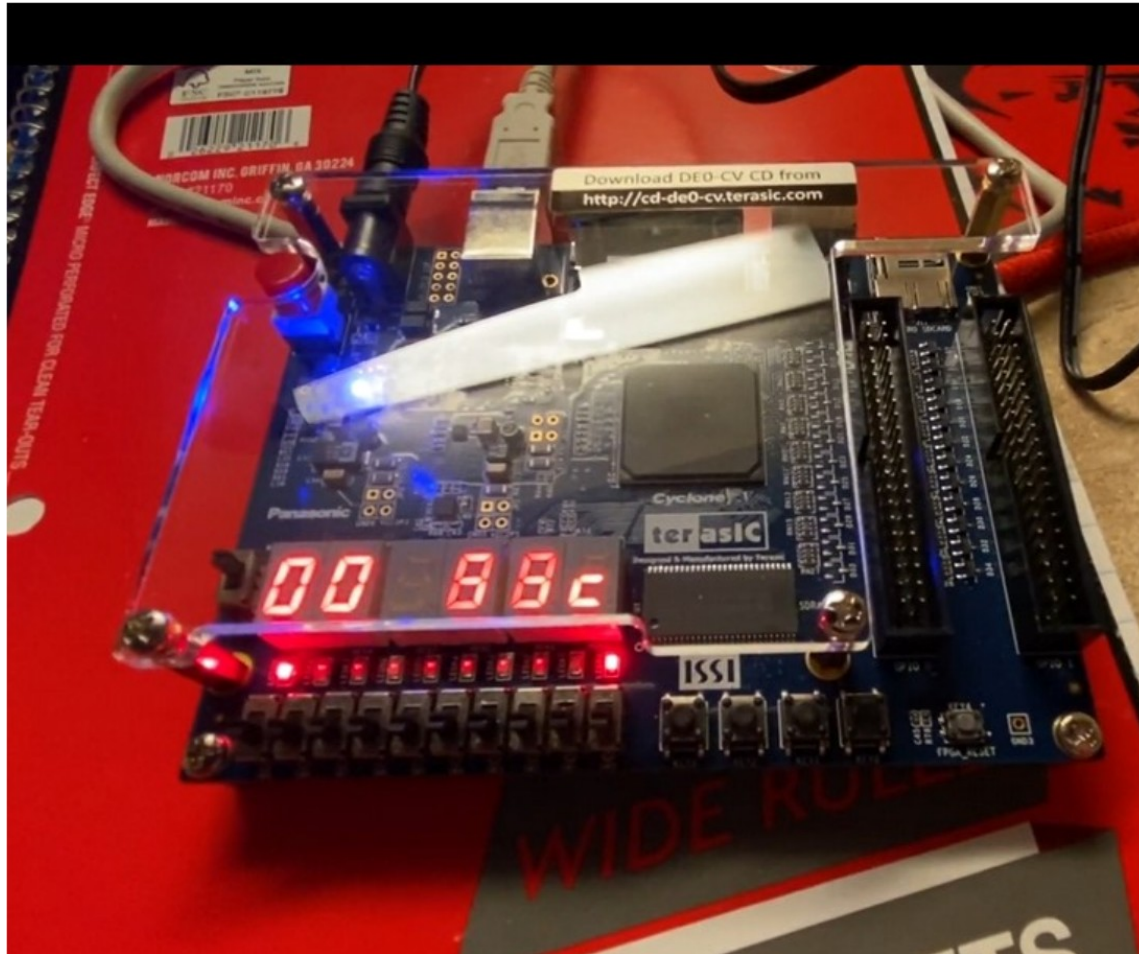


Figure 13: Current status is logged in after a round was over

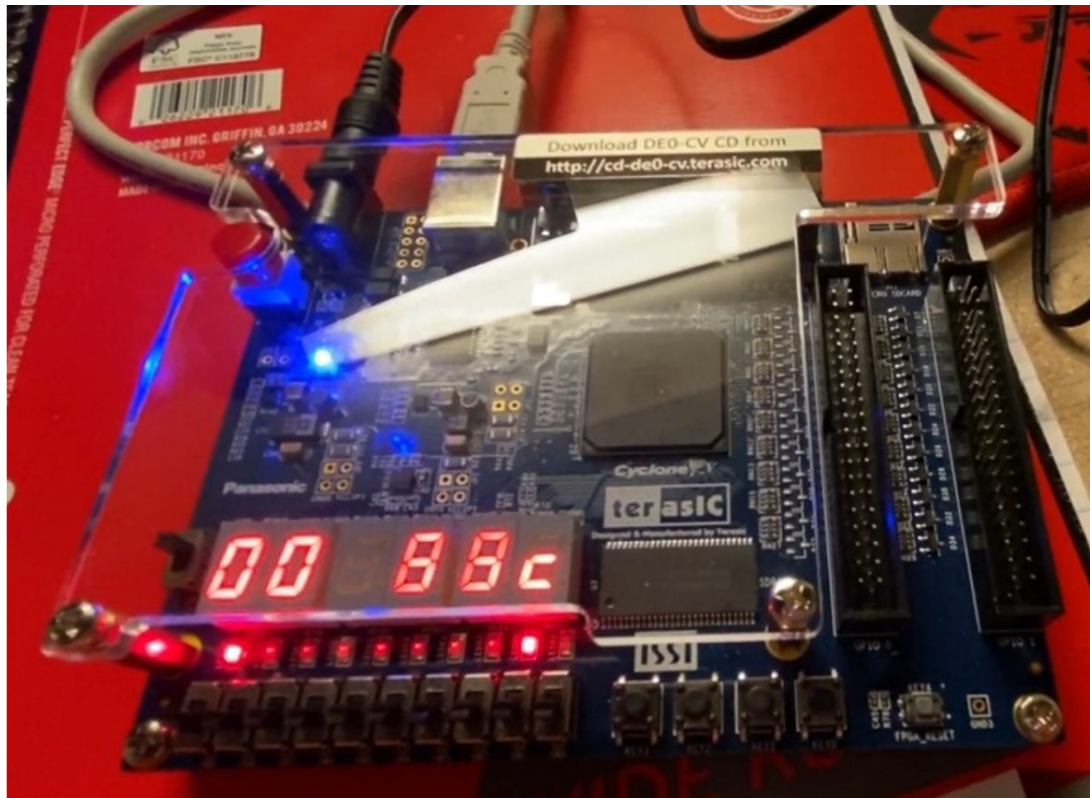


Figure 14: Current status is logged out after Player LOAD button was pressed

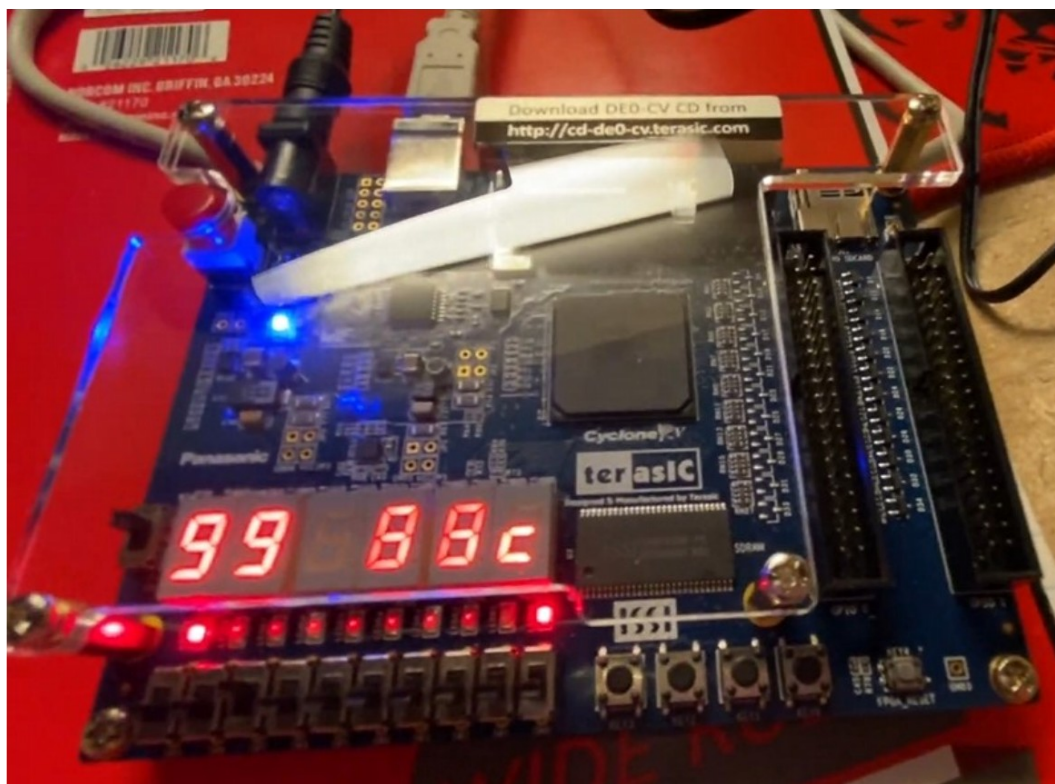


Figure 15: Current status is logged in after successfully entered password one more time

5. Video Demo

Bonus features' demo link: <https://drive.google.com/file/d/1mlQBHAreFLfd-cDZSDO7zERestcTUBec/view?usp=sharing>

Lab demo link: https://drive.google.com/file/d/1TJ--W3R9ITn4GCRFZ8ayViBRluNT_v_F/view?usp=sharing

6. Conclusion

The issue of the ones digit of the timer rolling over to 9 from 1 was still active as well as RNG displays number 8 outside of gameplay time. Besides that, this project was near successfully designed, tested, and implemented on the DE0-CV FPGA board. The bonus feature of Lab 1 of lighting respective LEDs to indicate matching or non-matching conditions was also maintained. The bonus features of Lab 4 (log out function and LFSR-based RNG) were also successfully implemented.

7. Appendix

```
module Lab4_NGUYEN_Tru(PNum, Pwd_Digit, B_in_P, B_in_Pwd_G,
B_RNG, clk, reset, Decoder_out_P, D_RNG_out, D_Sum_out,
D_ones_out, D_tens_out, M_LED_top, NM_LED_top, LoggedIn,
LoggedOut);
    input [3:0] PNum, Pwd_Digit;
    input B_in_P, B_in_Pwd_G, B_RNG, clk, reset;
    output [6:0] Decoder_out_P, D_RNG_out, D_Sum_out, D_ones_out,
D_tens_out;
    output M_LED_top, NM_LED_top, LoggedIn, LoggedOut;

    wire [3:0] Sum, P_LR_out, Rand_Num, tens_digit, ones_digit;
    wire Pwd_GameStart_Restart_Shaped, Load_P_in, Load_P_out,
RNG_Gen_out, timer_reconfig, timer_enable, timeout;

    SevenSegDecoder P_Decoder (P_LR_out, Decoder_out_P);
    SevenSegDecoder Sum_Decoder (Sum, D_Sum_out);
    SevenSegDecoder RNG_Decoder (Rand_Num, D_RNG_out);
    SevenSegDecoder ones_Decoder (ones_digit, D_ones_out);
    SevenSegDecoder tens_Decoder (tens_digit, D_tens_out);

    FourBitAdder FourBit_Adder (P_LR_out, Rand_Num, Sum);
    |
    LED_OnOff LED_Check (Sum, M_LED_top, NM_LED_top);

    ButtonShaper P_ButtonShaper (B_in_P, Load_P_in, clk, reset);
    //
    ButtonShaper Pwd_Game_BS (B_in_Pwd_G,
Pwd_GameStart_Restart_Shaped, clk, reset);//

    LoadRegister P_LoadReg (PNum, P_LR_out, clk, reset,
Load_P_out);//

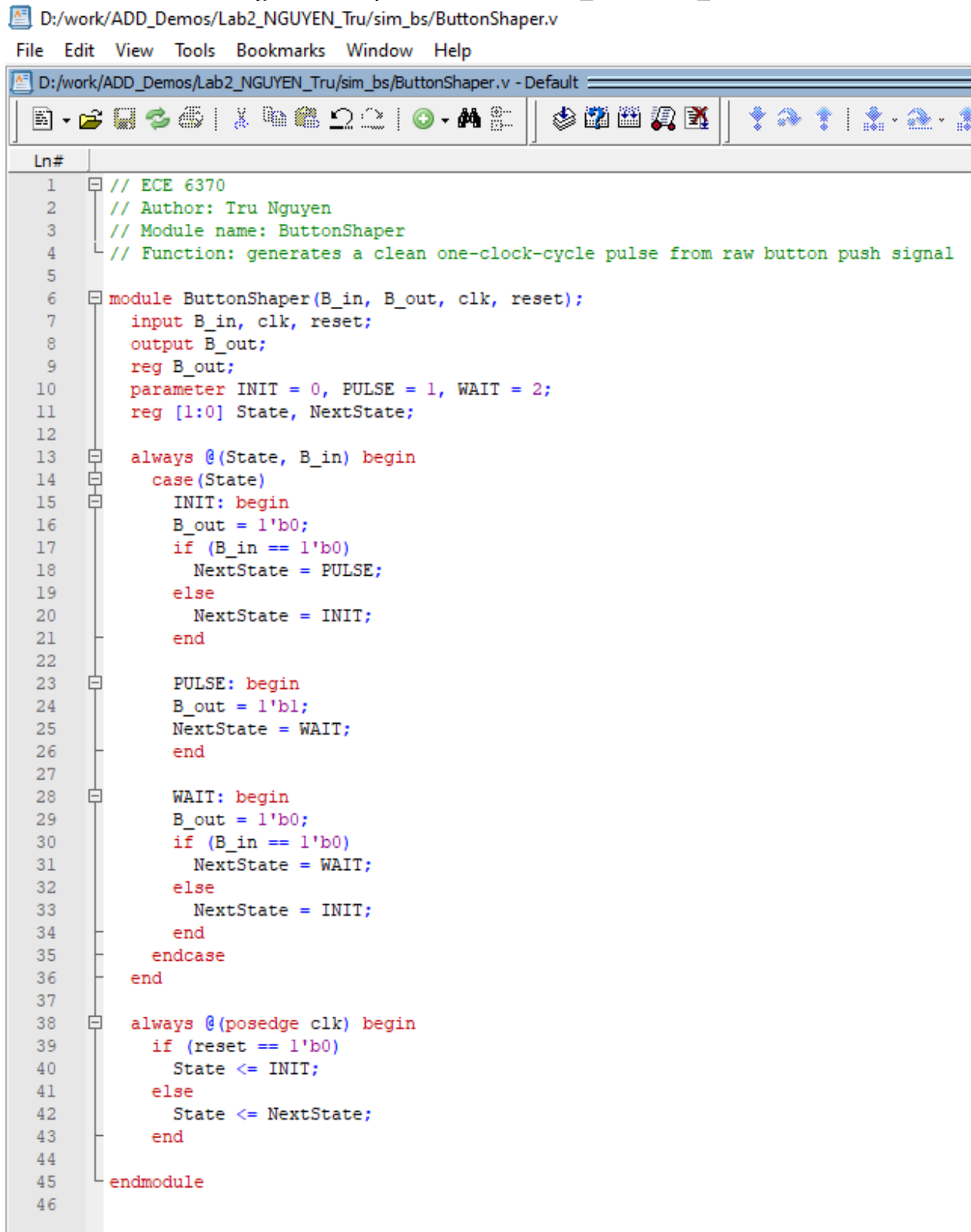
    TwoDigitTimer Two_digit_timer_top (timer_reconfig,
timer_enable, clk, reset, timeout, ones_digit, tens_digit);

    LFSR_RNG RNG_top (RNG_Gen_out, clk, reset, Rand_Num);//

    AccessController Access_top (Pwd_GameStart_Restart_Shaped,
Pwd_Digit, Load_P_in, B_RNG, timeout, clk, reset, LoggedIn,
LoggedOut, Load_P_out, RNG_Gen_out, timer_reconfig,
timer_enable);

endmodule
```


Figure 16: Top-level module Lab4_NGUYEN_Tru



The screenshot shows a Verilog code editor window titled "D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_bs/ButtonShaper.v - Default". The editor contains the following code:

```
1 // ECE 6370
2 // Author: Tru Nguyen
3 // Module name: ButtonShaper
4 // Function: generates a clean one-clock-cycle pulse from raw button push signal
5
6 module ButtonShaper(B_in, B_out, clk, reset);
7     input B_in, clk, reset;
8     output B_out;
9     reg B_out;
10    parameter INIT = 0, PULSE = 1, WAIT = 2;
11    reg [1:0] State, NextState;
12
13    always @(State, B_in) begin
14        case(State)
15            INIT: begin
16                B_out = 1'b0;
17                if (B_in == 1'b0)
18                    NextState = PULSE;
19                else
20                    NextState = INIT;
21            end
22
23            PULSE: begin
24                B_out = 1'b1;
25                NextState = WAIT;
26            end
27
28            WAIT: begin
29                B_out = 1'b0;
30                if (B_in == 1'b0)
31                    NextState = WAIT;
32                else
33                    NextState = INIT;
34            end
35        endcase
36    end
37
38    always @(posedge clk) begin
39        if (reset == 1'b0)
40            State <= INIT;
41        else
42            State <= NextState;
43        end
44    end
45 endmodule
46
```

Figure 17: The button shaper module

D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_bs/testbench_ButtonShaper.v

File Edit View Tools Bookmarks Window Help

D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_bs/testbench_ButtonShaper.v - Default

```

Ln#
1 // ECE 6370
2 // Author: Tru Nguyen
3 // Module name: testbench_ButtonShaper
4 // Function: a testbench to observe waveforms and verify the design of ButtonShaper module
5
6 `timescale 1 ns/100 ps
7
8 module testbench_ButtonShaper();
9     reg B_in, clk, reset;
10    wire B_out;
11
12    always                // artificial clk
13    begin
14        clk = 1'b0;
15        #10;
16        clk = 1'b1;
17        #10; // every cycle is 20ns long: 10ns high & 10ns low
18    end
19
20    ButtonShaper DUT_ButtonShaper(B_in, B_out, clk, reset);
21
22    initial
23    begin
24        B_in = 1'b1; // button is not pressed initially
25        reset = 1'b1; // reset button is not pressed
26
27        @ (posedge clk);
28        @ (posedge clk);
29        #5 reset = 1'b0;
30        @ (posedge clk);
31        @ (posedge clk);
32        @ (posedge clk);
33        @ (posedge clk);
34        #5 reset = 1'b1;
35        @ (posedge clk);
36        @ (posedge clk);
37        #5 B_in = 1'b0;
38        @ (posedge clk);
39        @ (posedge clk);
40        @ (posedge clk);
41        @ (posedge clk);
42        @ (posedge clk);
43        @ (posedge clk);
44        @ (posedge clk);
45        @ (posedge clk);
46        @ (posedge clk);
47        @ (posedge clk);
48        #5 B_in = 1'b1;
49
50        @ (posedge clk);
51        @ (posedge clk);
52        @ (posedge clk);
53        @ (posedge clk);
54        @ (posedge clk);
55        @ (posedge clk);
56        @ (posedge clk);
57        @ (posedge clk);
58        #5 reset = 1'b0;
59        @ (posedge clk);

```

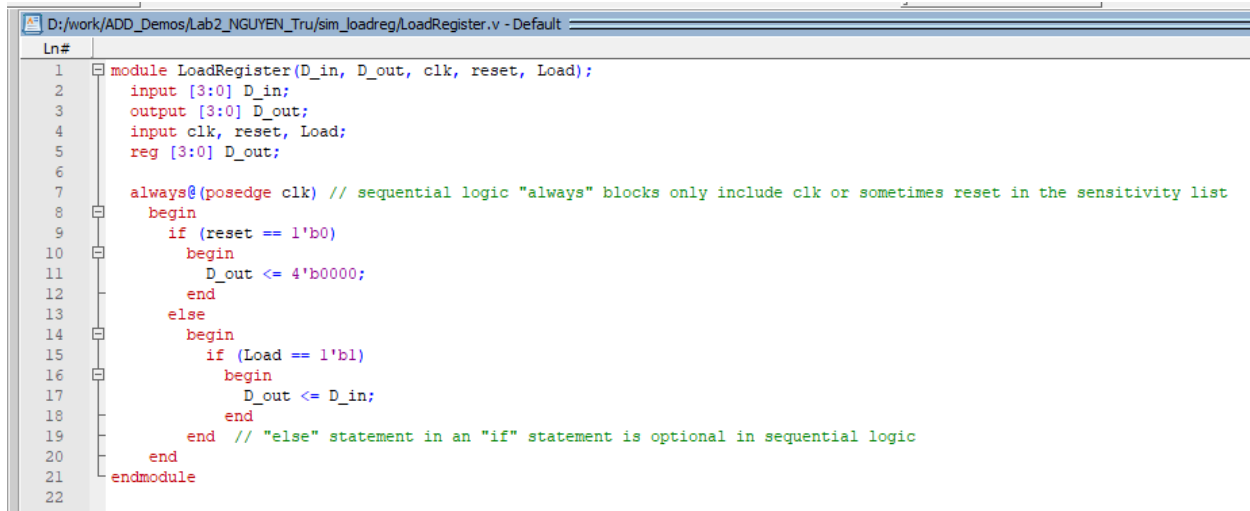
Figure 18: The button shaper testbench module part 1

```

49
50     @(posedge clk);
51     @(posedge clk);
52     @(posedge clk);
53     @(posedge clk);
54     @(posedge clk);
55     @(posedge clk);
56     @(posedge clk);
57     @(posedge clk);
58     #5 reset = 1'b0;
59     @(posedge clk);
60     @(posedge clk);
61     @(posedge clk);
62     @(posedge clk);
63     #5 reset = 1'b1;
64     @(posedge clk);
65     @(posedge clk);
66     #5 B_in = 1'b0;
67     @(posedge clk);
68     @(posedge clk);
69     @(posedge clk);
70     @(posedge clk);
71     @(posedge clk);
72     @(posedge clk);
73     @(posedge clk);
74     @(posedge clk);
75     @(posedge clk);
76     @(posedge clk);
77     #5 B_in = 1'b1;
78
79
80
81     end
82
83     endmodule
84

```

Figure 19: The button shaper testbench module part 2



The screenshot shows a Verilog code editor window titled "D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_loadreg/LoadRegister.v - Default". The code is for a module named "LoadRegister" with inputs "D_in", "D_out", "clk", "reset", and "Load". The output is "D_out". The code is as follows:

```
Ln#
1 module LoadRegister(D_in, D_out, clk, reset, Load);
2   input [3:0] D_in;
3   output [3:0] D_out;
4   input clk, reset, Load;
5   reg [3:0] D_out;
6
7   always@(posedge clk) // sequential logic "always" blocks only include clk or sometimes reset in the sensitivity list
8   begin
9       if (reset == 1'b0)
10      begin
11          D_out <= 4'b0000;
12      end
13      else
14      begin
15          if (Load == 1'b1)
16          begin
17              D_out <= D_in;
18          end
19      end // "else" statement in an "if" statement is optional in sequential logic
20  end
21 endmodule
22
```

Figure 20: The load register module

```

D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_loadreg/testbench_LoadRegister.v - Default
Ln#
1 // ECE 6370
2 // Author: Tru Nguyen
3 // Module name: testbench_LoadRegister
4 // Function: a testbench to observe signal waveforms and verify the design of LoadRegister module
5
6 `timescale 1 ns/100 ps
7
8 module testbench_LoadRegister();
9     reg clk, reset, Load;
10    reg [3:0] D_in;
11    wire [3:0] D_out;
12
13    always                // artificial clk
14    begin
15        clk = 1'b0;
16        #10;
17        clk = 1'b1;
18        #10; // every cycle is 20ns long: 10ns high & 10ns low
19    end
20
21    LoadRegister DUT_LoadRegister(D_in, D_out, clk, reset, Load);
22
23    initial
24    begin
25        reset = 1'b1; // button is not pressed at begin (reset = 0 means pressed; reset = 1 means not pressed)
26        Load = 1'b0;
27        D_in = 4'b0000;
28
29        @(posedge clk); // skip the first clk rising edge
30        @(posedge clk); // skip the second clk rising edge
31        #5 reset = 1'b0; // pull reset low 5 ns after the second clk rising edge/button pressed
32        @(posedge clk);
33        @(posedge clk);
34        @(posedge clk);
35        @(posedge clk);
36        #5 reset = 1'b1; // pull reset high after 4 and a half clk rising edge/button released
37        @(posedge clk);
38        @(posedge clk);
39        @(posedge clk);
40        @(posedge clk);
41        D_in = 4'b1010;
42        @(posedge clk);
43        Load = 1'b1; // pull Load high meaning D_out is rdy for data in D_in to be loaded in
44        @(posedge clk);
45        Load = 1'b0; // pull Load low after 1 clk rising edge
46        @(posedge clk);
47        D_in = 4'b0110;
48        @(posedge clk);
49        @(posedge clk);
50        D_in = 4'b1110;
51        Load = 1'b1;
52        @(posedge clk);
53        Load = 1'b0;
54        @(posedge clk);
55    end
56
57 endmodule
58

```

Figure 21: The load register testbench module

```

// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A design of a 7-segment decoder
// This module takes one 4-bit binary input signal decoder_in
// and produces a 7-bit output signal decoder_out that
// displays the corresponding input value in hexadecimal on the
// FPGA

module SevenSegDecoder(decoder_in, decoder_out);
    input [3:0] decoder_in;
    output [6:0] decoder_out;
    reg [6:0] decoder_out;

    always @(decoder_in)
    begin
        case(decoder_in)
            4'b0000: begin decoder_out = 7'b1000000; end
            4'b0001: begin decoder_out = 7'b1111001; end
            4'b0010: begin decoder_out = 7'b0100100; end
            4'b0011: begin decoder_out = 7'b0110000; end
            4'b0100: begin decoder_out = 7'b0011001; end
            4'b0101: begin decoder_out = 7'b0010010; end
            4'b0110: begin decoder_out = 7'b0000010; end
            4'b0111: begin decoder_out = 7'b1111000; end
            4'b1000: begin decoder_out = 7'b0000000; end
            4'b1001: begin decoder_out = 7'b0010000; end
            4'b1010: begin decoder_out = 7'b0001000; end
            4'b1011: begin decoder_out = 7'b0000011; end
            4'b1100: begin decoder_out = 7'b0100111; end
            4'b1101: begin decoder_out = 7'b0100001; end
            4'b1110: begin decoder_out = 7'b0000110; end
            4'b1111: begin decoder_out = 7'b0001110; end
            default: begin decoder_out = 7'b1111111; end
        endcase
    end

endmodule

```

Figure 22: The 7-segment decoder module


```

`timescale 1 ns/100 ps
module testbench_SevenSegDecoder();

    reg [3:0] decoder_in_s;
    wire [6:0] decoder_out_s;

    SevenSegDecoder DUT_SevenSegDecoder(decoder_in_s,
decoder_out_s);

    initial
        begin
            decoder_in_s = 4'b0000;
            #10 decoder_in_s = 4'b0001;
            #10 decoder_in_s = 4'b0010;
            #10 decoder_in_s = 4'b0011;
            #10 decoder_in_s = 4'b0100;
            #10 decoder_in_s = 4'b0101;
            #10 decoder_in_s = 4'b0110;
            #10 decoder_in_s = 4'b0111;
            #10 decoder_in_s = 4'b1000;
            #10 decoder_in_s = 4'b1001;
            #10 decoder_in_s = 4'b1010;
            #10 decoder_in_s = 4'b1011;
            #10 decoder_in_s = 4'b1100;
            #10 decoder_in_s = 4'b1101;
            #10 decoder_in_s = 4'b1110;
            #10 decoder_in_s = 4'b1111;
            end
endmodule

```

Figure 23: The 7-segment decoder testbench module

```

// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A design of a 4-bit binary adder
// This module takes two 4-bit binary input signals P1Num and
// P2Num, calculating the sum of the input signals, and store
// the 4-bit binary result to the output signal Sum

module FourBitAdder(P1Num, P2Num, Sum);
    input [3:0] P1Num, P2Num;
    output [3:0] Sum;
    reg [3:0] Sum;

    always @(P1Num, P2Num)
    begin
        Sum = P1Num + P2Num;
    end
endmodule

```

Figure 24: The 4-bit adder module

```

`timescale 1 ns/100 ps

module testbench_FourBitAdder();

    reg [3:0] P1Num_s, P2Num_s;
    wire [3:0] Sum_s;

    FourBitAdder DUT_FourBitAdder(P1Num_s, P2Num_s, Sum_s);

    initial
    begin
        P1Num_s = 4'b1010; P2Num_s = 4'b0101; // matching case:
        10+5=15
        #10 P1Num_s = 4'hC; P2Num_s = 4'h1; // non-matching case:
        12+1=13
        #10 P1Num_s = 4'd11; P2Num_s = 4'd6; // overflow: 11+6=
        17 -> (1) 0001

    end
endmodule

```

Figure 25: The 4-bit adder testbench module

```

// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A verification design to check if the sum of two 4-
bit binary numbers equals F
// This module takes one 4-bit binary input signal Sum, checks
if Sum equals F, if so, light up the left-most LED, otherwise
light up the right-most LED.

module LED_OnOff(Sum, M_LED, NM_LED);
    input [3:0] Sum;
    output M_LED, NM_LED;
    reg M_LED, NM_LED;

    always @(Sum, M_LED, NM_LED)
        begin
            if (Sum == 4'b1111)
                begin
                    M_LED = 1'b1;
                end
            else
                begin
                    NM_LED = 1'b1;
                end
        end
    end
endmodule

```

Figure 26: The verification module of matching and non-matching conditions


```

// ECE 6370
// Author: Tru Nguyen
// Module name: countTo10
// Function: counts 100 ms timeout pulses and generates a 1
// second timeout signal. Once the count limit is reached, it
// resets and asserts OneSecTimeOut for one clock cycle.
// (Note: Counter size is reduced for simulation purposes.)

module countTo10(HundredMsTimeOut, clk, reset, OneSecTimeOut,
Count_out);
    input HundredMsTimeOut, clk, reset;
    output reg OneSecTimeOut;
    output reg [3:0] Count_out;
    //output [1:0] Count_out; // for sim only
    //reg OneSecTimeOut;
    //reg [3:0] Count_out;
    //reg [1:0] Count_out; // for sim only

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Count_out <= 4'd0;
            OneSecTimeOut <= 1'b0;
        end
        else begin
            if (HundredMsTimeOut == 1'b1) begin
                if (Count_out == 4'd10) begin
                    Count_out <= 4'd0;
                    OneSecTimeOut <= 1'b1;
                end
            end
            else begin
                Count_out <= Count_out+1;
                OneSecTimeOut <= 1'b0;
            end
        end
        else begin
            OneSecTimeOut <= 1'b0;
        end
    end
end

endmodule

```

Figure 27: The countTo10 module

```

// ECE 6370
// Author: Tru Nguyen
// Module name: countTo100
// Function: counts incoming 1 ms timeout pulses and generates a
100 ms timeout signal. When the count reaches its limit, it
resets and produces a one-cycle output pulse.
// (Note: Counter size is reduced for simulation purposes.)

module countTo100(OneMsTimeOut, clk, reset, HundredMsTimeOut,
Count_out);
    input OneMsTimeOut, clk, reset;
    output reg HundredMsTimeOut;
    output reg [6:0] Count_out;
    //output [1:0] Count_out; // for sim only
    //reg HundredMsTimeOut;
    //reg [6:0] Count_out;
    //reg [1:0] Count_out; // for sim only

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Count_out <= 7'd0;
            HundredMsTimeOut <= 1'b0;
        end
        else begin
            if (OneMsTimeOut == 1'b1) begin
                if (Count_out == 7'd100) begin
                    Count_out <= 7'd0;
                    HundredMsTimeOut <= 1'b1;
                end
            end
            else begin
                Count_out <= Count_out+1;
                HundredMsTimeOut <= 1'b0;
            end
        end
        else begin
            HundredMsTimeOut <= 1'b0;
        end
    end
end

endmodule

```

Figure 28: The countTo100 module

```

module digitTimer(reconfig, BorrowDN, NoBorrowUp, clk, reset,
Digit, BorrowUp, NoBorrowDN);
    input reconfig, BorrowDN, NoBorrowUp, clk, reset;
    output [3:0] Digit;
    output BorrowUp, NoBorrowDN;

    reg [3:0] Digit;
    reg BorrowUp, NoBorrowDN;

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Digit <= 4'd0;
            BorrowUp <= 1'b0;
            NoBorrowDN <= 1'b1;
        end

        else begin
            BorrowUp <= 1'b0;

            if (reconfig == 1'b0) begin

                if (BorrowDN == 1'b1) begin

                    if (Digit == 4'd1) begin

                        if (NoBorrowUp == 1'b0) begin
                            Digit <= 4'd9;
                            BorrowUp <= 1'b1;
                            NoBorrowDN <= 1'b0;
                        end // NoBorrowUp

                        else begin
                            BorrowUp <= 1'b0;
                            Digit <= 4'd0;
                            NoBorrowDN <= 1'b1;
                        end // NoBorrowUp else

                    end // Digit

                else begin
                    Digit <= Digit - 1;
                    NoBorrowDN <= 1'b0;
                    // BorrowUp <= 1'b0;
                    // NoBorrowDN <= 1'b0;
                end // Digit else

            end // BorrowDN

        else begin
            if (Digit == 4'd0) begin
                NoBorrowDN <= 1'b1;
            end

            else begin

```

Figure 29: The digitTimer module part 1

```

        end // BorrowDN

        else begin
            if (Digit == 4'd0) begin
                NoBorrowDN <= 1'b1;
            end

            else begin
                NoBorrowDN <= 1'b0;
            end

        end // BorrowDN else

        end // reconfig

        else begin
            Digit <= 4'd9;
            BorrowUp <= 1'b0;
            NoBorrowDN <= 1'b0;

            end // reconfig else

        end // reset else
    end // always
endmodule

```

Figure 30: The digitTimer module part 2


```

// ECE 6370
// Author: Tru Nguyen
// Module name: OneMsTimer
// Function: generates a 1 millisecond timeout pulse by counting
clock cycles when enabled. When the internal counter reaches its
maximum value, it resets and asserts OneMsTimeOut for one clock
cycle.

module OneMsTimer(enable, clk, reset, OneMsTimeOut, Count_out);
    input enable, clk, reset;
    output reg OneMsTimeOut;
    output reg [15:0] Count_out;
    //output [2:0] Count_out; // for sim only
    //reg OneMsTimeOut;
    //reg [15:0] Count_out;
    //reg [2:0] Count_out; // for sim only

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Count_out <= 16'd0;
            OneMsTimeOut <= 1'b0;
        end
        else begin
            if (enable == 1'b1) begin
                if (Count_out == 16'd50000) begin
                    Count_out <= 16'd0;
                    OneMsTimeOut <= 1'b1;
                end
                else begin
                    Count_out <= Count_out+1;
                    OneMsTimeOut <= 1'b0;
                end
            end
        end
    end
end

endmodule

```

Figure 31: The OneMsTimer module

```

| // ECE 6370
// Author: Tru Nguyen
// Function: A clocked 16-bit LFSR-based timer. When enable=1,
it advances the 16-bit LFSR state every posedge clk.
// When the LFSR reaches the terminal count 9363, it asserts
timeout signal for one cycle
// and resets LFSR back to 0 to start the timing interval again.

module OneMsLFSR(enable, clk, reset, OneMsTimeOut);
    input enable, clk, reset;
    output reg OneMsTimeOut;

    reg [15:0] LFSR;
    wire feedback = LFSR[15];

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            LFSR <= 16'd0;
            OneMsTimeOut <= 1'b0;
        end
        else begin
            OneMsTimeOut <= 1'b0;
            if (enable == 1'b1) begin
                if (LFSR == 16'd37449) begin
                    LFSR <= 16'd0;
                    OneMsTimeOut <= 1'b1;
                end

                else begin
                    LFSR[0] <= feedback;
                    LFSR[1] <= LFSR[0];
                    LFSR[2] <= LFSR[1] ^ feedback;
                    LFSR[3] <= LFSR[2] ^ feedback;
                    LFSR[4] <= LFSR[3];
                    LFSR[5] <= LFSR[4] ^ feedback;
                    LFSR[6] <= LFSR[5];
                    LFSR[7] <= LFSR[6];
                    LFSR[8] <= LFSR[7];
                    LFSR[9] <= LFSR[8];
                    LFSR[10] <= LFSR[9];
                    LFSR[11] <= LFSR[10];
                    LFSR[12] <= LFSR[11];
                    LFSR[13] <= LFSR[12];
                    LFSR[14] <= LFSR[13];
                    LFSR[15] <= LFSR[14];
                end
            end
        end
    end
endmodule

```

Figure 32: The OneMsLFSR module

```

module LFSR_Counter(enable, clk, reset, Rand_out);
    input enable, clk, reset;
    output [3:0] Rand_out;

    reg [3:0] LFSR;

    wire feedback = LFSR[3];

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            LFSR <= 4'b0000;
        end

        else begin
            if (enable == 1'b1) begin
                LFSR[0] <= feedback;
                LFSR[1] <= LFSR[0] ^ feedback;
                LFSR[2] <= LFSR[1];
                LFSR[3] <= LFSR[2];
            end

        end

        end

    assign Rand_out = LFSR;

endmodule

```

Figure 33: The LFSR_Counter module

```

// ECE 6370
// Author: Tru Nguyen
// Function: This is a simulation testbench that generates
stimulus to test timing behavior.

`timescale 1 ns/100 ps

module testbench_OneMsLFSR();
    reg enable, clk, reset;
    wire OneMsTimeOut, OneMsTimeOut_LFSR;

    always
    begin
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    OneMsTimer DUT_OneMsTimer(enable, clk, reset, OneMsTimeOut);
    OneMsLFSR DUT_OneMsLFSR(enable, clk, reset,
OneMsTimeOut_LFSR);

    initial begin
        reset = 1'b1; // not pressed
        enable = 1'b0;

        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b0; // reset is pressed
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b1; // released
        @(posedge clk);
        @(posedge clk);
        #5 enable = 1'b1; // start counting
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);

    end
endmodule

```

Figure 34: The OneMsLFSR testbench module


```

// ECE 6370
// Author: Tru Nguyen
// Module name: OneSecTimer
// Function: the top-level timing module that generates a 1-
// second timeout signal by cascading three smaller timers.
// It uses a 1 ms timer, then counts those pulses to form 100
// ms, and finally counts again to produce a 1-second pulse.

module OneSecTimer(enable, clk, reset, OneSecTimeOut);
    input enable, clk, reset;
    output OneSecTimeOut;
    wire OneMsTimeOut, HundredMsTimeOut;

    OneMsLFSR OneMsLFSR_top (enable, clk, reset, OneMsTimeOut);

    countTo100 count_To_100(OneMsTimeOut, clk, reset,
HundredMsTimeOut);

    countTo10 count_To_10(HundredMsTimeOut, clk, reset,
OneSecTimeOut);

endmodule

```

Figure 35: The OneSecTimer module

```

// Author: Tru Nguyen
// Module name: testbench_OneSecTimer
// Function: A simulation testbench that verifies the
functionality of the OneSecTimer module by generating a clock
signal and applying reset and enable stimuli to observe the
OneSecTimeOut output.

`timescale 1 ns/100 ps

module testbench_OneSecTimer();
    reg clk, reset, enable;
    wire OneSecTimeOut;

    always
    begin
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    OneSecTimer DUT_OneSecTimer(enable, clk, reset,
OneSecTimeOut);

    initial
    begin
        reset = 1'b1; // not pressed
        enable = 1'b0;

        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b0; // reset is pressed
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b1; // released
        @(posedge clk);
        @(posedge clk);
        #5 enable = 1'b1; // start counting
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);

    end

endmodule

```

Figure 36: The OneSecTimer testbench module

```

module TwoDigitTimer(reconfig, timer_enable, clk, reset,
timeout, ones_digit_out, tens_digit_out);
    input reconfig, timer_enable, clk, reset;
    output [3:0] ones_digit_out, tens_digit_out;
    output timeout;

    wire BorrowUp_OnesToTens, BorrowUp_Tens;
    wire NoBorrowDN_Ones, NoBorrowDN_Tens; // BorrowUp_Tens is
left floating; NoBorrowUp_Tens is wired to logic high 1'b1
    wire OneSecTimeOut;

    digitTimer Ones_Digit(reconfig, OneSecTimeOut,
NoBorrowDN_Tens, clk, reset, ones_digit_out,
BorrowUp_OnesToTens, NoBorrowDN_Ones);

    digitTimer Tens_Digit(reconfig, BorrowUp_OnesToTens, 1'b1,
clk, reset, tens_digit_out, BorrowUp_Tens, NoBorrowDN_Tens);

    OneSecTimer One_Sec_Timer(timer_enable, clk, reset,
OneSecTimeOut);

    assign timeout = NoBorrowDN_Ones;
endmodule

```

Figure 37: The TwoDigitTimer module

```

`timescale 1 ns/100 ps

module testbench_TwoDigitTimer();
    reg reconfig, timer_enable, clk, reset;
    wire timeout;
    wire [3:0] ones_digit_out, tens_digit_out;

    always
    begin
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    TwoDigitTimer DUT_TwoDigitTimer(reconfig, timer_enable, clk,
reset, timeout, ones_digit_out, tens_digit_out);

    initial
    begin
        reset = 1'b1; // not pressed
        timer_enable = 1'b0;
        reconfig = 1'b0;

        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b0; // reset is pressed
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b1; // released
        @(posedge clk);
        @(posedge clk);
        #5 reconfig = 1'b1;
        @(posedge clk);
        #5 reconfig = 1'b0;
        @(posedge clk);
        @(posedge clk);
        #5 timer_enable = 1'b1;

    end
endmodule

```

Figure 38: The TwoDigitTimer testbench module


```

module AccessController(Pwd_GameStart_Restart_Shaped, Pwd_digit,
Load_P_in, B_RNG, timeout, clk, reset, LoggedIn, LoggedOut,
Load_P_out, RNG_Gen_out, timer_reconfig, timer_enable);
    input Pwd_GameStart_Restart_Shaped, Load_P_in, B_RNG, timeout,
clk, reset;
    input [3:0] Pwd_digit;
    output LoggedIn, LoggedOut, Load_P_out, RNG_Gen_out,
timer_reconfig, timer_enable;

    wire Passed, Logout_Req;

    Authentication Top_Auth(Pwd_digit,
Pwd_GameStart_Restart_Shaped, Logout_Req, clk, reset, LoggedIn,
LoggedOut, Passed);
    GameController Top_GameCtrl(Pwd_GameStart_Restart_Shaped,
B_RNG, Load_P_in, timeout, Passed, clk, reset, Load_P_out,
RNG_Gen_out, timer_reconfig, timer_enable, Logout_Req);

endmodule

```

Figure 39: The AccessController module

```

module GameController(B_GamePlay_Shaped, B_RNG, Load_P_in,
timeout, Passed, clk, reset, Load_P_out, RNG_Gen_out,
timer_reconfig, timer_enable, Logout_Req);

    input B_GamePlay_Shaped, B_RNG, Load_P_in, timeout, Passed,
clk, reset;

    output reg Load_P_out, RNG_Gen_out, timer_reconfig,
timer_enable, Logout_Req;

    parameter INIT = 0, RECONFIGTIMER = 1, WAITFORGAMESTART = 2,
GAMEPLAY = 3, GAMEOVER = 4, LOGOUTWAIT = 5;

    reg [2:0] State;

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Load_P_out <= 1'b0;
            RNG_Gen_out <= 1'b0;
            timer_reconfig <= 1'b0;
            timer_enable <= 1'b0;
            Logout_Req <= 1'b0;
            State <= INIT;
        end

        else begin
            case(State)
                INIT: begin
                    Load_P_out <= 1'b0;
                    RNG_Gen_out <= 1'b0;
                    timer_reconfig <= 1'b0;
                    timer_enable <= 1'b0;
                    Logout_Req <= 1'b0;
                    if (B_GamePlay_Shaped == 1'b1)begin
                        State <= INIT;
                    end

                    if (Passed == 1'b1) begin
                        State <= RECONFIGTIMER;
                    end

                    else begin
                        State <= INIT;
                    end

                    RECONFIGTIMER: begin
                        Load_P_out <= 1'b0;
                        RNG_Gen_out <= 1'b0;
                        timer_reconfig <= 1'b1;
                        timer_enable <= 1'b0;

                        if (Load_P_in == 1'b1) begin
                            Logout_Req <= 1'b1;
                            State <= INIT;
                        end
                    end
                end
            endcase
        end
    end

```

Figure 40: The GameController module part 1

```

        State <= INIT;
    end

    else begin
        State <= WAITFORGAMESTART;
    end
end

WAITFORGAMESTART: begin
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b0;
    timer_reconfig <= 1'b0;

    if (Load_P_in == 1'b1) begin
        Logout_Req <= 1'b1;
        State <= INIT;
    end

    else begin
        if (B_GamePlay_Shaped == 1'b1) begin
            timer_enable <= 1'b1;
            State <= GAMEPLAY;
        end

        else begin
            timer_enable <= 1'b0;
            State <= WAITFORGAMESTART;
        end
    end
end

GAMEPLAY: begin
    Load_P_out <= Load_P_in;
    RNG_Gen_out <= B_RNG;
    timer_enable <= 1'b1;
    timer_reconfig <= 1'b0;
    Logout_Req <= 1'b0;

    if (timeout == 1'b1) begin
        State <= GAMEOVER;
    end

    else begin
        State <= GAMEPLAY;
    end
end

end

GAMEOVER: begin
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b0;
    timer_reconfig <= 1'b0;
    timer_enable <= 1'b0;

    if (Load_P_in == 1'b1) begin
        Logout_Req <= 1'b1;
    end
end

```

Figure 41: The GameController module part 2

```

GAMEOVER: begin
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b0;
    timer_reconfig <= 1'b0;
    timer_enable <= 1'b0;

    if (Load_P_in == 1'b1) begin
        Logout_Req <= 1'b1;
        State <= LOGOUTWAIT;
    end

    else begin
        if (B_GamePlay_Shaped == 1'b1) begin
            State <= RECONFIGTIMER;
        end

        else begin
            State <= GAMEOVER;
        end
    end
end

    LOGOUTWAIT: begin
        Load_P_out <= 1'b0;
        RNG_Gen_out <= 1'b0;
        timer_reconfig <= 1'b0;
        timer_enable <= 1'b0;
        Logout_Req <= 1'b0;
        State <= INIT;
    end

    default: begin
        Load_P_out <= 1'b0;
        RNG_Gen_out <= 1'b0;
        timer_reconfig <= 1'b0;
        timer_enable <= 1'b0;
        Logout_Req <= 1'b0;

        if (Passed == 1'b1) begin
            State <= RECONFIGTIMER;
        end

        else begin
            State <= INIT;
        end
    end

endcase
end
endmodule

```

Figure 42: The GameController module part 3


```

module Authentication(Pwd_digit, B_Pwd_shaped, Logout_Req, clk,
reset, LoggedIn, LoggedOut, Passed);
    input [3:0] Pwd_digit;
    input B_Pwd_shaped, Logout_Req, clk, reset;

    output LoggedIn, LoggedOut, Passed;

    wire [4:0] ROM_in;
    wire [3:0] ROM_out;

    ROM_Authentication Top_ROM_Auth(Pwd_digit, B_Pwd_shaped,
ROM_out, Logout_Req, clk, reset, ROM_in, LoggedIn, LoggedOut,
Passed);
    ROM_Password Top_ROM_Pwd(ROM_in, clk, ROM_out);

endmodule

```

Figure 43: The Authentication module

```

module ROM_Authentication(P_digit, B_PW_shaped, ROMdigit,
Logout_Req, clk, reset, ROMAddr, LoggedIn, LoggedOut, Passed);
    input B_PW_shaped, Logout_Req, clk, reset;
    input [3:0] P_digit, ROMdigit;

    output reg LoggedIn, LoggedOut, Passed;
    output reg [4:0] ROMAddr;

    reg [3:0] P_digit_reg, ROMdigit_reg;
    reg GoodUntilNow;
    reg [1:0] Counter;
    reg [3:0] State;

    parameter BUTTONCHECK = 0, FETCHROM = 1, ROMWAIT1 = 2,
ROMWAIT2 = 3;
    parameter ROMLOAD = 4, COMPARE = 5, CHECKCOUNT = 6, VERIFY =
7, PASSED = 8;

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            LoggedIn <= 1'b0;
            LoggedOut <= 1'b1;
            Passed <= 1'b0;
            GoodUntilNow <= 1'b1;
            ROMAddr <= {3'b000, Counter};
            Counter <= 2'b00;
            State <= BUTTONCHECK;
        end

        else begin
            case(State)
                BUTTONCHECK: begin
                    LoggedIn <= 1'b0;
                    LoggedOut <= 1'b1;
                    Passed <= 1'b0;

                    if (B_PW_shaped == 1'b1) begin
                        P_digit_reg <= P_digit;
                        State <= FETCHROM;
                    end

                end

            else begin
                State <= BUTTONCHECK;
            end

        end

        FETCHROM: begin
            LoggedIn <= 1'b0;
            LoggedOut <= 1'b1;
            Passed <= 1'b0;
            ROMAddr <= {3'b000, Counter};
            State <= ROMWAIT1;
        end
    end

```

Figure 44: The ROM_Authentication module part 1

```

ROMWAIT1: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Passed <= 1'b0;
    State <= ROMWAIT2;
end

ROMWAIT2: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Passed <= 1'b0;
    State <= ROMLOAD;
end

ROMLOAD: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Passed <= 1'b0;
    ROMdigit_reg <= ROMdigit;
    State <= COMPARE;
end

COMPARE: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Passed <= 1'b0;

    if (P_digit_reg == ROMdigit_reg) begin

    end

    else begin
        GoodUntilNow <= 1'b0;
    end
    State <= CHECKCOUNT;
end

CHECKCOUNT: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Passed <= 1'b0;

    if (Counter == 2'b11) begin
        State <= VERIFY;
    end

    else begin
        Counter <= Counter + 2'b01;
        State <= BUTTONCHECK;
    end

end

VERIFY: begin // DIGIT1 state

```

Figure 45: The ROM_Authentication module part 2

```

VERIFY: begin // DIGIT1 state
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Passed <= 1'b0;

    if (GoodUntilNow == 1'b1) begin
        State <= PASSED;
    end

else begin
    State <= BUTTONCHECK;
    GoodUntilNow <= 1'b1;
    Counter <= 2'b00;
    end

end

PASSED: begin
    LoggedIn <= 1'b1;
    LoggedOut <= 1'b0;
    Passed <= 1'b1;

    if (Logout_Req == 1'b1) begin
        State <= BUTTONCHECK;
        Counter <= 2'b00;
        Passed <= 1'b0;
    end

    else begin
        State <= PASSED;
    end

end

default: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Passed <= 1'b0;
    GoodUntilNow <= 1'b1;
    Counter <= 2'b00;
    ROMAddr <= {3'b000, Counter};

    if (B_PW_shaped == 1'b1) begin
        P_digit_reg <= P_digit;
        State <= FETCHROM;
    end

    else begin
        State <= BUTTONCHECK;
    end

end

endcase

```

Figure 46: The ROM_Authentication module part 3


```

module LFSR_RNG(RNG_Gen, clk, reset, Rand_Num);
    input RNG_Gen, clk, reset;
    output [3:0] Rand_Num;

    wire enable;
    wire [3:0] Rand_out;

    LFSR_Counter LFSR_Counter_top(enable, clk, reset, Rand_out);

    assign enable = ~RNG_Gen;
    assign Rand_Num = Rand_out;

endmodule

```

Figure 47: The LFSR_RNG module

```

module LFSR_Counter(enable, clk, reset, Rand_out);
    input enable, clk, reset;
    output [3:0] Rand_out;

    reg [3:0] LFSR;

    wire feedback = LFSR[3];

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            LFSR <= 4'b0000;
        end

        else begin
            if (enable == 1'b1) begin
                LFSR[0] <= feedback;
                LFSR[1] <= LFSR[0] ^ feedback;
                LFSR[2] <= LFSR[1];
                LFSR[3] <= LFSR[2];
            end

        end

        end

    assign Rand_out = LFSR;

endmodule

```

Figure 48: The LFSR_counter module

```

`timescale 1 ns/ 100 ps

module testbench_LFSR_RNG();
    reg RNG_Gen, clk, reset;

    wire [3:0] Rand_Num;

    always begin // clock
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    LFSR_RNG DUT_LFSR_RNG(RNG_Gen, clk, reset, Rand_Num);

    initial begin
        reset = 1'b1;
        RNG_Gen = 1'b1;

        @(posedge clk);
        @(posedge clk);

        #5 reset = 1'b0;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b1;

        @(posedge clk);
        @(posedge clk);
        #5 RNG_Gen = 1'b0;

        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5 RNG_Gen = 1'b1;

    end
endmodule

```

Figure 49: The LFSR_RNG testbench module